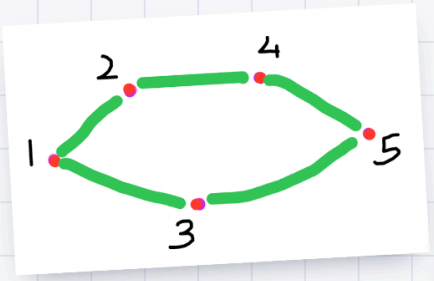
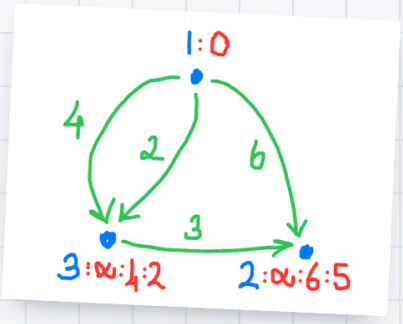
FEN LİSESİ İÇİN C++ VE ALGORİTMA

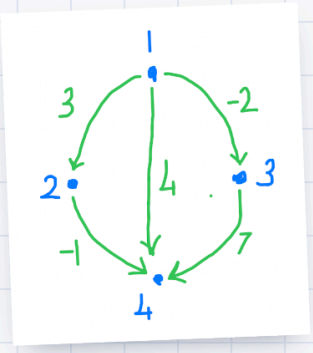
Programlamaya ve Algoritmalara Keyifli Bir Başlangıç

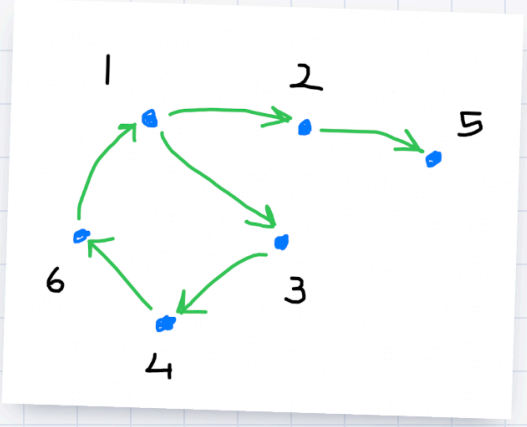
Altı bölüm, yirmi dört çalışan program ve bir sürü soru.
Yalnız da okunur, ama ikili daha hızlı.

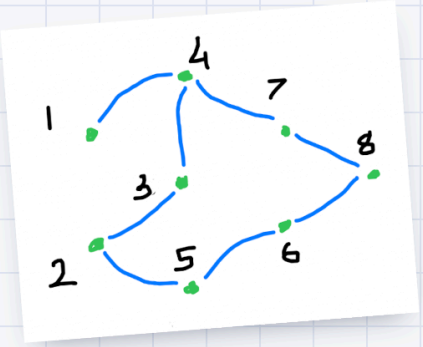
I İlk Adımlar · II Veri · III Zanaat · IV Özyineleme · V Çizgeler · VI Dinamik Programlama











derste tahtaya çizdiklerimizden

Bülent Başaran

2026 · birinci sürüm

CC BY-SA 4.0 · kod MIT

github.com/bulent2k2/cpp_ogreniyoruz

FEN LİSESİ İÇİN C++ VE ALGORİTMA KİTAPÇIĞI

Programlamaya ve Algoritmalara Keyifli Bir Başlangıç

Bu kitapçık iki dönemlik ders notlarımızın, birlikte yazdığımız programların ve sorduğunuz soruların damıtılmış hâli. Amacı size programlamayı *anlatmak* değil; kendi kendinize öğrenebileceğinizi *fark ettirmek*. Bir de arkadaşınızla yan yana oturunca işin ne kadar hızlandığını. Kitapçığın bir de kardeşi var: aynı yolu Koco ve Scala ile yürüyen *Keyifli ve İşlevsel Bir Giriş*.

I. İlk Adımlar · II. Veriyi Düzenlemek · III. Zanaat · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

ÖNSÖZ

Merhaba

Programlamayı öğrenmenin en zor tarafı, ilk birkaç haftanın insana “ben bunu beceremem” dedirtmesi. Noktalı virgül unutulur, derleyici anlaşılmaz bir hata verir, program çalışır ama yanlış sayı yazar. Sonra bir gün bir şey tıklar: bilgisayarın aslında son derece *aptal* ama son derece *itaatkâr* olduğunu anlarsınız. O günden sonrası keyif.

Bu kitapçık o günü öne çekmek için yazıldı. İçindeki her kavram, derslerimizde gerçekten yazdığımız, çalıştırdığımız ve bazen de hata verdiği için birlikte düzelttiğimiz programlardan geliyor. Hataları da sakladım, çünkü hata bulmak programcılığın yarısı.

Altı bölüm var. İlk üçü temeli kuruyor: dil, veri ve zanaat. Son üçü algoritmalara ayrılmış: özyineleme, çizgeler ve dinamik programlama. Sırayla okumanız gerekmiyor ama sırayla okursanız her bölüm bir öncekine yaslanacak.

YÖNTEM

Nasıl çalışalım?

Altı ilke. Hepsi derslerde pahalı yollardan öğrendiğimiz şeyler; hiçbiri kitaptan okunmadı.

1. **Kod okumadan kod yazılmaz.** Müzisyen dinleyerek, yazar okuyarak öğrenir. Siz de başkasının kodunu satır satır okuyun. Anlamadığınız satırda durun, tahmin edin, sonra çalıştırıp tahmininizi sınayın. Bu kitapçıktaki her program okunmak için var.

2. **Çalıştırmadan inanmayın.** Bir kodun ne yapacağını "herhâlde şunu yapar" diye geçiştirmeyin. Çalıştırın. Bilgisayar elinizin altında; dünyanın en sabırlı öğretmeni ve hiç yorulmuyor.

3. **Önce doğru, sonra hızlı.** Programcıların meşhur bir sözü var: *bütün baş belalarının kaynağı sırasız yapılan iyileştirmelerdir.* Çalışan yavaş bir program, çalışmayan hızlı bir programdan sonsuz kere iyidir.

4. **Kanıt isteyin.** Kodunuz doğru yanıtı verdi diye doğru olduğunu bilmiyorsunuz; sadece o girdide yanlış olmadığını biliyorsunuz. Neden çalıştığını anlatabiliyor musunuz? Anlatamıyorsanız, henüz çözmediniz demektir. Bilmediğimizi bilmek de bir bilgidir; bilmemek ayıp değil.

5. **Yapay zekâya danışın, ama güvenmeyin.** Çok iyi bir yol arkadaşı, çok kötü bir hakem. Yazdığı kodu okuyun, çalıştırın, sınayın. Derslerimizde yapay zekânın yazdığı kodlarda birden fazla hata bulduk.

6. **Yarım bıraktığım yerler kasıtlı.** Bu kitapçıkta bilerek tamamlanmamış işlevler ve yanıtı verilmemiş sorular var. Onlar boşluk değil, davet. Bir çözümü okuyarak öğrenilenle, bir çözümü bulmaya çalışırken öğrenilen aynı şey değil.

> **PÜF NOKTASI**
>
> Takıldığınızda geçirdiğiniz süreyi ölçün. Yirmi dakika ilerleyemediyseniz, problemi bir arkadaşınıza *yüksek sesle* anlatın. Anlatırken çözdüğünüz çok olacak. Buna "lastik ördek yöntemi" deniyor: karşınızdaki dinlemese de olur, hatta ördek bile olur.

TAKIM

Yalnız gidebilirsiniz, ama ikili daha hızlı

Bu kitapçığın en önemli sayfası burası. Programlamayı tek başınıza öğrenebilirsiniz — buna hiç şüphem yok, insanlar öğreniyor. Ama iki ya da üç kişilik küçük takımlarla çalışanların ne kadar hızlı yol aldığını her dönem görüyorum. Nedeni de basit: **programlamanın zor kısmı yazmak değil, düşünmek.** Düşünmek de tek başına yapılınca kısır döngüye girmeye çok müsait.

Yazılım dünyasında bunun bir adı var: *ikili programlama* (*pair programming*). Kuralları şöyle:

Sürücü

Klavye onda. Sadece o yazar. Aklından geçeni sesli söyler: "şimdi döngüyü kuruyorum, sayaç sıfırdan başlasın…"

Yol gösterici

Klavyeye dokunmaz. Bir adım ileriyi düşünür: "dizinin dışına taşacak galiba", "bunun testi ne olacak?"

Üçüncü kişi

Varsa test yazar ve karşı örnek arar. Takımın en eğlenceli işi: arkadaşlarının kodunu kırmaya çalışmak.

- **On beş dakikada bir rol değişin.** Zamanlayıcı kurun. Değişmezseniz hızlı olan yazar, yavaş olan seyreder ve kimse öğrenmez.
- **Önce algoritmayı konuşun, sonra klavyeye dokunun.** Kâğıt kalem alın, küçük bir örneği elle çözün. Elle çözemediğiniz bir şeyi bilgisayara anlatamazsınız.
- **"Anlamadım" demek takımın en değerli cümlesi.** Anlamamış gibi görünmemek için susan bir takım, hep birlikte yanlış yola sapar.
- **Aynı problemi ayrı ayrı çözüp sonra kodları karşılaştırmak** da harika bir yöntem. İki farklı çözüm görmek, tek çözümü iki kere görmekten çok daha öğretici.

TAKIM İŞİ

Bu kitapçıkta yeşil kutular takımca yapılacak işleri gösteriyor. İlki şu: bir araya gelip **haftada bir sabit saat** belirleyin ve o saati koruyun. Düzenli bir saat, uzun ama düzensiz saatlerden çok daha verimli. İki saatlik haftalık bir buluşma, bir dönemde sizi şaşırtacak bir yere getirir.

KARDEŞ

İki kitapçık, tek yol

Bu kitapçığın bir kardeşi var: Programlamaya ve Algoritmalara Keyifli ve İşlevsel Bir Giriş — aynı konuları Koco (Kojo'nun Türkçesi) ve Scala ile anlatıyor. Orada derleyici yok, bekleme yok: kaplumbağaya komut verirsiniz, tuvalde çizer. Bölümleri bilerek paralel kurdum: buradaki sayı tahmin oyunu orada da var; buradaki Hanoi kuleleri orada Koch tanesi oluyor; buradaki çizge gezisi orada işlevsel kılıkta karşınıza çıkıyor.

Hangisinden başlamalı? C++ ile başladıysanız buradan devam edin; ama bir bölümü bitirince oradaki eşine göz atın. Aynı algoritmayı iki dilde görmek, tek dilde iki kere görmekten çok daha öğretici — tıpkı iki farklı çözümü karşılaştırmanın tek çözümü ezberlemekten öğretici olması gibi.

PÜF NOKTASI

Bir bölümü burada bitirip kardeş kitapçıktaki eşine bakınca kendinize şu soruyu sorun: *hangi kısım orada daha kolaydı, hangisi burada?* Bu sorunun yanıtı size dillerin karakterini, dillerin karakteri de programlamanın kendisini öğretir.

Tezgâhınızı kurun

Marangozun tezgâhı, programcının da geliştirme ortamı var. İki aşamada kurun: önce hiçbir şey yüklemeden başlayın, sonra kendi bilgisayarınıza inin.

Bugün, hiçbir şey yüklemeden

Tarayıcıda çalışan derleyiciler var. İlk haftalar için fazlasıyla yeterli, hatta kod paylaşmak için uzun süre en pratik yol:

SİTE	İYİ TARAFI	NOT
Sololearn	Sade, hızlı açılıyor	`std::cin >> girdi;` çalışıyor
OnlineGDB	Girdi ve çıktı altta, aynı terminalde	Hata ayıklayıcısı (*debugger*) var, çok işe yarar
Coliru	Derleme komutunu siz yazıyorsunuz	Sağ alttaki **Edit** tuşuna basın

Bu ay, kendi bilgisayarınızda

Er ya da geç kendi makinenizde derlemeniz gerekecek; büyük testler tarayıcıda zorlanıyor. Gereken üç şey var: bir **terminal**, bir **derleyici** ve bir **editör**.

- **Windows:** Cygwin kurun, içine `gcc-g++` paketini ekleyin. Cygwin terminalinde `pwd` yazınca `/home/<kullanıcı>` görmelisiniz. Alternatif olarak WSL de çok iyi çalışıyor.
- **macOS:** terminalde `c++ --version` yazın. Yoksa Xcode komut satırı araçlarını kurmayı önerecektir; kabul edin.
- **Linux:** zaten hazır. Değilse dağıtımınızın paket yöneticisiyle `g++` kurun.
- **Editör:** VS Code her yerde çalışıyor ve kolay. Emacs eski ve dik başlı ama bir kere alışınca bırakılmıyor. Hangisi olduğu önemli değil, birini seçip kısayollarını öğrenmeniz önemli.

terminal — İLK SINAV

```
# derleyici duruyor mu?
$ c++ --version

# ders deposunu indirin
$ git clone https://github.com/bulent2k2/cpp_ogreniyoruz
$ cd cpp_ogreniyoruz
$ ls

# tek dosyalık bir programı derleyip çalıştırın
$ c++ -std=c++23 ilkProgram.cpp -o ilk
$ ./ilk
```

Hesaplarınızı açın

Bunların hepsi ücretsiz ve hepsi bir dönem boyunca işinize yarayacak. Bugün açın, sonra unutulur:

- GitHub — kodlarınızı saklamak ve paylaşmak için
- Project Euler — matematik kokan, tadına doyulmaz sorular
- CSES Problem Set — algoritma öğrenmek için sıraya dizilmiş en iyi liste
- USACO — ABD bilgisayar olimpiyatı ön elemeleri, deneme sınavları
- Codeforces ve AtCoder — yarışmalar

Bu kitapçıkta neler var?

BÖLÜM	KONU	SONUNDA YAPABİLECEĞİNİZ
I. İlk Adımlar	Değer, değişken, tür, işlev; koşullar ve döngüler; kapsam	Kullanıcıyla konuşan bir tahmin oyunu ve ilk Project Euler çözümünüz
II. Veriyi Düzenlemek	Dizi, akıllı dizi, eşlem, küme, yığın, kuyruk, öncelik sırası; kendi türleriniz; kalıplar	Bir metindeki harfleri sayan, kendi türünü tanımlayan programlar
III. Zanaat	Derleme, `make`, testler, hata ayıklama, hız ölçümü	Kendini sınayan, tek komutla derlenen bir proje düzeni
IV. Özyineleme ve Arama	Kendini çağıran işlevler, bellekle hızlandırma, geri dönüşlü arama, budama	Sekiz vezir bulmacası ve altküme/permütasyon üretimi
V. Çizgeler	Derinlemesine ve enlemesine gezi, bağlı parçalar, Dijkstra, Floyd–Warshall, Bellman–Ford	Labirentte en kısa yol, canavarlardan kaçış, en ucuz uçuş rotası
VI. Dinamik Programlama	Problemi küçültmek, tümevarım formülü kurmak, yukarıdan aşağı ve aşağıdan yukarı	Bozuk para ve zar dizisi problemlerini kendi başınıza kurup çözmek

SON BİR UYARI

Bu kitapçıktaki **alıştırmaların çözümleri yok** ve bu bir eksiklik değil. Bazı örnek kodlar da bilerek yarım bırakıldı. Takıldığınız yerde ipucu var, ama yanıt yok. Yanıt sizde — ya da yan masadaki arkadaşınızda.

BAŞKA BİÇİMLERDE

Kitapçığın tamamı ders deposunda iki dosya hâlinde de duruyor:

- **PDF** — altmış dört sayfa, A4'e basılmak üzere düzenlendi: `kitap/…-Baslangic.pdf`
- **EPUB** — telefon, tablet ve e-kitap okuyucular için akışkan sürüm: `kitap/…-Baslangic.epub`

İçindeki bütün programların kaynak dosyaları da `kitapcik/kod/` dizininde, derlenmeye hazır.

Kaynak: 2024–25 ve 2025–26 dönemi ders notları, birlikte yazdığımız programlar ve sınıfta sorulan sorular. Ders notlarının tamamı GitHub deposunda, dizini de `ileri/icindekiler.md` dosyasında. Kardeş kitapçığın kaynakları, PDF'i ve EPUB'ı da aynı depoda: `kitapcik-koco/`.

KÜNYE

Telif ve lisans

Programlamaya ve Algoritmalara Keyifli Bir Başlangıç
Bülent Başaran · 2026 · Birinci sürüm

Bu kitapçığın **metni** Creative Commons Atıf-AynıLisanslaPaylaş 4.0 (CC BY-SA 4.0) lisansıyla sunuluyor. Kopyalayın, basın, dağıtın, çevirin, değiştirin — ticari olarak da. Tek istediğimiz kaynağı anmanız ve türev çalışmanızı da aynı lisansla paylaşmanız.

İçindeki **bütün örnek programlar** ise MIT lisansı altında. Yani hiçbir yükümlülük altına girmeden kendi projelerinize kopyalayabilirsiniz. Zaten onun için yazıldılar.

ATIF

“Programlamaya ve Algoritmalara Keyifli Bir Başlangıç”, Bülent Başaran, CC BY-SA 4.0. github.com/bulent2k2/cpp_ogreniyoruz

Kitapçıkta anılan ve bağlantı verilen dış kaynaklar — Project Euler, CSES, USACO, Codeforces, AtCoder ve Antti Laaksonen'in *Rekabetçi Programlama El Kitabı* — kendi sahiplerine ve kendi koşullarına tabidir. Ders deposundaki üçüncü taraf malzemenin dökümü `LICENSE` dosyasında.

Bu kitapçıktaki bütün programlar `g++ -std=c++23 -Wall` ile derlenip çalıştırıldı; gösterilen çıktılar gerçek çıktılardır. Bir hata bulursanız ya da bir bölümün daha iyi anlatılabileceğini düşünüyorsanız, deposunda konu açın — kitapçık da ders gibi, birlikte gelişiyor.

BÖLÜM I

İlk Adımlar

Bir programın içinde aslında çok az şey var: değerler, onlara verdiğimiz adlar, kararlar, tekrarlar ve işler. Beşi de bu bölümde. Sonunda bilgisayarla oyun oynayacak ve ilk Project Euler sorunuzu çözeceksiniz.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

I.1

Değer, ad, tür

Programlamanın tamamı şu üçlünün etrafında dönüyor. Bir **değer** var: `16`. Bu değere bir **ad** veriyoruz: `yas`. Ve bilgisayara bu değerin ne cins bir şey olduğunu, yani **türünü** söylüyoruz: `int`.

Neden tür söylemek zorundayız? Çünkü bilgisayarın belleği, üzerine sadece sıfır ve bir yazılabilen upuzun bir şerit. Aynı sıfır-bir dizisi, türüne göre bambaşka şeyler demek: bir sayı, bir harf, bir doğru/yanlış. Tür, o şeridin nasıl okunacağını söyleyen etiket. C++ bu etiketi *derleme* anında istiyor — işte hızının sırrı da burada.

01-turler.cpp — TEMEL TÜRLER

```cpp
#include <iostream>
using namespace std;

int main() {
  cout << "Merhaba! Ben senin ilk programın." << endl;

  int    yas      {16};        // sayı
  double boy      {1.72};      // kesirli sayı
  char   basHarfi {'B'};       // harf
  bool   ogrenci  {true};      // ikil

  cout << "Yaş      : " << yas      << endl;
  cout << "Boy      : " << boy      << " m" << endl;
  cout << "Baş harfi: " << basHarfi << endl;
  cout << "Öğrenci  : " << ogrenci  << endl;            // 1 yazar!
  cout << "Öğrenci  : " << boolalpha << ogrenci << endl; // true yazar

  cout << "On yıl sonra: " << yas + 10 << endl;
  return 0;
}
```

Süslü parantezle ilk değer verme (`int yas{16}`) alışkanlığını baştan edinin. `int yas = 16` da çalışır ama süslü parantez, veri kaybına yol açan sessiz dönüşümlere derleyicinin itiraz etmesini sağlar. **İlk değeri vermeyi asla unutmayın:** ilk değeri verilmemiş bir değişkende bellekte o an ne varsa o durur; çoğu zaman sıfır çıkar ve sizi kandırır, bir gün `1696521216` çıkar ve saatinizi alır.

> **DENEYİN**
>
> `ogrenci` değişkenini `false` yapın. Çıktı ne olur? Sonra `char basHarfi{'B'}` yerine `int basHarfi{'B'}` yazıp çalıştırın. Çıkan `66` sayısı nereden geldi? (İpucu: harfler de bellekte sayı olarak duruyor. Adı *ASCII*.)

Her türün bir sınırı var

Şimdi çok öğretici bir hataya bakalım. Aşağıdaki programın çıktısını okumadan önce tahmin edin:

02-tasma.cpp — TAŞMA

```cpp
#include <iostream>
#include <cmath>
#include <limits>
using namespace std;

int main() {
  int a{2000}, b{3000};

  cout << "2000'in küpü (int) : " << a * a * a << endl;   // eksi!
  cout << "3000'ün küpü (int) : " << b * b * b << endl;   // yanlış!
  cout << "3000'ün küpü (long long): " << 1LL * b * b * b << endl;

  cout << "karekök(2000'in küpü) : " << sqrt(a * a * a) << endl; // nan

  cout << "int'in en büyüğü                : "
       << numeric_limits<int>::max() << endl;
  cout << "unsigned long long'un en büyüğü : "
       << numeric_limits<unsigned long long>::max() << endl;
  return 0;
}
```

çıktı — GERÇEK SONUÇ

```
2000'in küpü (int) : -589934592
3000'ün küpü (int) : 1230196224
3000'ün küpü (long long): 27000000000
karekök(2000'in küpü) : -nan
int'in en büyüğü                : 2147483647
unsigned long long'un en büyüğü : 18446744073709551615
```

Bir sayının küpü nasıl eksi olur? Olmaz elbette — ama `int` türü yaklaşık iki milyar iki yüz milyona kadar sayabiliyor. Bunu aşınca sayaç başa dönüyor, tıpkı kilometre saati gibi. Buna **taşma** (*overflow*) deniyor. Eksi bir sayının karekökü de haliyle `nan`, yani "sayı değil" (*not a number*). Bir uyarı: C++ işaretli bir sayı taşınca ne olacağını *tanımlamaz* (*undefined behavior*); gördüğünüz, bu derleyicinin bu seferki kararı. Başka bir derleyici, hatta aynı derleyicinin `-O2` bayrağı, başka bir şey yapabilir.

Kilometre saati benzetmesi yalnızca `unsigned` türler için kesin: onlarda başa dönmek kuralın kendisidir.

Bu hata, yarışma sorularında en çok can yakan hatadır: kodunuz küçük örneklerde mükemmel çalışır, büyük testte sessizce yanlış yanıt verir. **Çarpım yapıyorsanız türünüzü büyütün** — `long long` kullanın ya da `1LL *` ile çarpıma başlayın. (`long long` her yerde 64 bit; `long` Windows'ta 32 bit kalır, ona güvenmeyin.) Son satırdaki `unsigned long long` sınırına bakın: 18 milyar kere milyardan fazla. Neredeyse her şeye yeter.

> **TAKIM İŞİ**
>
> İkiniz de kâğıt kalemle şunu hesaplayın: `int` 32 bit ise, en büyük değeri neden tam olarak `2147483647`? Neden `2147483648` değil? Sonra yanıtlarınızı karşılaştırın. Bu tek sorunun cevabı, sayıların bellekte nasıl durduğunu bir ömür hatırlatır.

I.2

Kararlar ve tekrarlar

Şimdiye kadarki program yukarıdan aşağıya bir kere aktı. Programı *program* yapan iki şey daha var: bazı satırları **duruma göre** çalıştırmak ve bazı satırları **defalarca** çalıştırmak.

kalıplar

```
// karar
if (not sayi) cout << "sıfır";
else if (sayi < 0) cout << "eksi";
else cout << "artı";

// sayısı belli tekrar
for (int k{0}; k < 10; ++k)
    cout << k << " ";

// sayısı belli olmayan tekrar
while (hak > 0) {
    // ...
    --hak;
}
```

C++ dilinde `and`, `or`, `not` sözcükleri `&&`, `||`, `!` imgelerinin birebir karşılığı. İkisini de tanıyın; sözcük hâli Türkçe kod yazarken çok daha okunaklı oluyor, ders notlarımızda genelde onları kullandık.

İşlev: bir kere yaz, çok kere kullan

Bir **işlev** (*function*) girdi alan, iş yapan ve çıktı veren küçük bir makine. Programı işlevlere bölmenin üç faydası var: okumak kolaylaşır, sınamak kolaylaşır ve aynı şeyi iki kere yazmaktan kurtulursunuz.

Şimdi bu üçünü birden kullanan bir çözüme bakalım. Project Euler'in birinci sorusu şunu istiyor: 1000'den küçük, 3'e ya da 5'e tam bölünen bütün sayıların toplamı kaçtır?

04-pe1.cpp — PROJECT EULER 1

```cpp
#include <iostream>
using namespace std;

bool bolunuyorMu(int sayi, int bolen) {
  return sayi % bolen == 0;
}

int main() {
  int toplam{0};
  for (int sayi{3}; sayi < 1000; ++sayi)
    if (bolunuyorMu(sayi, 3) or bolunuyorMu(sayi, 5))
      toplam += sayi;

  cout << "Project Euler, birinci problem: " << toplam << endl;
  return 0;
}
```

```
$ ./pe1 → Project Euler, birinci problem: 233168
```

`bolunuyorMu()` işlevi tek satır. Buna değer mi? Kesinlikle değer. Adı sayesinde `if` satırı Türkçe bir cümle gibi okunuyor: “sayı üçe bölünüyorsa ya da beşe bölünüyorsa”. İyi kod, yorum satırına ihtiyaç bırakmayan koddur.

> **KLASİK TUZAK**
>
> Bu soruyu `if`'i ikiye bölerek yazarsanız — biri 3 için, biri 5 için — 15'e bölünen sayıları iki kere saymış olursunuz. Derste tam olarak bu hatayı yaptık ve `if (bolunuyorMu(sayi, 15)) toplam -= sayi;` diye bir satırla yamaladık. Çalışıyor, ama yukarıdaki `or`'lu hâli hem daha kısa hem daha doğru. **Yamamak yerine yeniden düşünmeyi** alışkanlık edinin.

I.3

İlk oyununuz

Şimdi bilgisayarla konuşan bir program yazalım. Bilgisayar bir sayı tutsun, siz tahmin edin, o da “daha büyük / daha küçük” desin.

03-tahmin.cpp — SAYI TAHMİN OYUNU

```cpp
#include <iostream>
#include <cstdlib>   // rand, srand
#include <ctime>     // time
using namespace std;

int main() {
  srand(time(nullptr));            // her çalıştırışta farklı olsun
  const int gizli{rand() % 100 + 1};  // 1..100
  const int toplamHak{7};
  int hak{toplamHak};

  cout << "1 ile 100 arasında bir sayı tuttum. "
       << hak << " hakkın var." << endl;

  while (hak > 0) {
    cout << "Tahminin: ";
    int tahmin;
    if (not (cin >> tahmin)) break;   // girdi bitti ya da bozuk
    --hak;

    if (tahmin == gizli) {
      cout << "Bildin! " << (toplamHak - hak) << " tahminde." << endl;
      return 0;
    }
    cout << (tahmin < gizli ? "Daha büyük." : "Daha küçük.")
         << " Kalan hak: " << hak << endl;
  }
  cout << "Hakkın bitti. Tuttuğum sayı " << gizli << " idi." << endl;
  return 0;
}
```

Küçük ama içinde çok şey var: rastgele sayı, kullanıcıdan girdi, döngü, koşul, üçlü işlemci (`? :`) ve bozuk girdiye karşı bir savunma. Bir de `const` var: `gizli` bir kere belirlenip bir daha değişmeyecek, biz de bunu derleyiciye söylüyoruz ki yanlışlıkla değiştirirsek bize haber versin.

Neden tam yedi hak?

Burası bu bölümün en güzel yeri. Rastgele bir sayı seçmiyoruz. Akıllı oyuncu her seferinde **ortayı** söyler: 50. “Daha büyük” derse geriye 50 sayı kalır; sonra 75, geriye 25… Her tahmin arama uzayını ikiye böler:

TAHMİN	1	2	3	4	5	6	7
Kalan olasılık	50	25	13	7	4	2	1

Yedi adımda kesin buluyoruz, çünkü $2^7 = 128 > 100$. Bu yönteme **ikili arama** (*binary search*) deniyor ve bu kitapçığın sonuna kadar karşınıza defalarca çıkacak. Milyon sayı arasından aradığınızı yirmi adımda bulur. Bir milyar arasından otuz adımda. İşte algoritma budur: aynı işi bambaşka bir hızla yapmak.

DENEYİN

Sınırı 100 yerine 1000 yapın. Kaç hak vermelisiniz? Programı, hak sayısını sınıra bakarak kendisi hesaplayacak biçimde değiştirin. (İpucu: `<cmath>` içinde `log2` var, ama önce `while` döngüsüyle elle hesaplamayı deneyin — daha öğretici.)

I.4

Adların nerede geçerli olduğu

Bir değişkenin adı her yerde tanınmıyor. Süslü parantezin içinde tanımlanan bir ad, o parantez kapanınca yok oluyor. Buna **kapsam** (*scope*) deniyor. İşlevin dışında, en üstte tanımlanan adlar ise **küresel**: her yerden görünüyorlar.

Küresel değişkenler pratiktir ama tehlikelidir; hangi işlevin ne zaman değiştirdiğini kestiremezsiniz. Sınav sorusu yazanlar da bunu çok sever. Şuna bakın, gerçek bir sınav sorusundan sadeleştirildi:

kapsam.cpp — NE YAZAR?

```
#include <iostream>
int sayi = 3;              // küresel

void f() { sayi = 10; }     // küreseli değiştirir
void g() { int sayi = 99; } // kendi yerelini yapar, küresele dokunmaz

int main() {
  int sayi = 5;            // yerel, küreseli gölgeliyor
  f();
  g();
  std::cout << sayi << " " << ::sayi;   // ?? ve ??
}
```

Yanıtı vermiyorum; çalıştırın. Sadece bir ipucu: `::sayi` yazımı “küresel olanı kastediyorum” demek. Bu tür sorular kötü kod örnekleridir ve **kasıtlı olarak** kafa karıştırmak için yazılırlar. Sizden beklenen, bu kodu yazmanız değil; okuduğunuzda tuzağı görmeniz.

Kendi programlarınızda kuralı basit tutun: **bir değişkeni, ihtiyaç duyulan en dar kapsamda tanımlayın.** Döngüde kullanılacaksa döngünün içinde, işlevde kullanılacaksa işlevin içinde.

I.5

Bölümü kapatırken

Bir programın beş yapıtaşını gördünüz: değer, ad, tür, karar, tekrar. Bir de altıncısını: bunları paketleyen işlev. Bundan sonrası, bu beş taşla giderek daha büyük binalar kurmaktan ibaret. Gerçekten öyle — bu kitapçığın son bölümündeki Bellman–Ford algoritması da bu beş taşla yazılıyor.

ALIŞTIRMALAR

1. **Tahmin oyununu tersine çevirin.** Siz bir sayı tutun, bilgisayar tahmin etsin, siz "büyük/küçük" deyin. Kaç adımda buluyor? Yukarıdaki tabloyla uyuşuyor mu?
2. **Fibonaççi.** Project Euler 2: dört milyonu aşmayan çift Fibonaççi sayılarının toplamı. Şimdilik döngüyle yazın; özyinelemeyi Bölüm IV'te göreceğiz.
3. **Asal çarpanlar.** Project Euler 3: `600851475143` sayısının en büyük asal çarpanı. TUZAK Bu sayı `int`'e sığmıyor. Hangi türü seçeceksiniz?
4. **Doğum günü sayacı.** Komut satırından yıl, ay, gün alıp "siz doğalı kaç gün olmuş" yazan bir program. Artık yılları unutmayın. (Depoda `gun-sayimi/` dizininde bir örnek var, ama önce kendiniz deneyin.)
5. **Kendi kendine sınanan program.** Yazdığınız `bolunuyorMu()` işlevini `<cassert>` içindeki `assert` ile sınayın: `assert(bolunuyorMu(15, 5));` Bir de yanlış olması gereken bir durumu sınayın. Bu alışkanlık Bölüm III'te büyüyecek.

TAKIM İŞİ

İkinci ve üçüncü alıştırmayı ikili programlamayla yapın. On beş dakika biri yazsın, on beş dakika öteki. Sonra kodu bir kere daha, bu sefer sıfırdan ve *ezberden* yazın. İkinci yazışın ne kadar hızlı olduğuna şaşıracaksınız; öğrenmenin gerçekten olduğu an orası.

BÖLÜM II

Veriyi Düzenlemek

Tek bir sayıyla ne kadar iş yapılabilir? Programların gücü, çok sayıda veriyi düzenli tutmaktan geliyor. Bu bölümde hazır kapları tanıyacak, sonra kendi türünüzü yazacaksınız — ve bir tür yazdığınız an dilin size çalıştığını hissedeceksiniz.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

II.1

Dizi: bellekte yan yana

En eski, en sade ve en hızlı kap: **dizi** (*array*). Aynı türden birkaç değer, bellekte yan yana. Öğelerin yerini hesaplamak bedava — ilk öğenin adresini biliyorsanız, üçüncü öğe “ilk adres + 2 × öğe boyu” adresindedir. Bu yüzden dizide arama değil ama *erişim* anlıktır.

10-dizi.cpp — BELLEKTE NE VAR?

```
#include <iostream>
using namespace std;

int main() {
  short dizi[]{3, 1, 4, 1, 5, 9, 2, 6};

  cout << "dizinin boyu     : " << sizeof(dizi)    << " bayt" << endl;
  cout << "bir öğenin boyu  : " << sizeof(dizi[0]) << " bayt" << endl;
  int ogeSayisi = sizeof(dizi) / sizeof(dizi[0]);
  cout << "öğe sayısı       : " << ogeSayisi << endl;

  cout << "ilk öğenin adresi: " << (void*) dizi       << endl;
  cout << "ikincininki      : " << (void*) (dizi + 1) << endl;

  for (auto s : dizi) cout << s << " ";
  cout << endl;

  cout << "taşma: " << dizi[12] << endl;   // dizinin dışı! kimse uyarmıyor
  return 0;
}
```

çıktı

```
dizinin boyu     : 16 bayt
bir öğenin boyu  : 2 bayt
öğe sayısı       : 8
ilk öğenin adresi: 0x7ffc4e4f2eb0
ikincininki      : 0x7ffc4e4f2eb2
3 1 4 1 5 9 2 6
taşma: -1536
```

İki adres arasındaki fark tam **2 bayt**: bir `short`'un boyu. Bellek gerçekten şerit gibi ve diziler o şeridin üzerinde bitişik duruyor.

> **DİKKAT**
>
> Son satıra bakın: `dizi[12]` diye bir öğe yok, ama program çökmedi — bellekte o adreste ne varsa onu okuyup yazdı. C++ sizi korumaz, size *güvenir*. Bu hem hızının hem de tehlikesinin kaynağı. Yarışmalarda "çalışıyor ama bazen yanlış" hatalarının çoğu budur. **Etrafı duvarla çevirmek** — yani diziyi bir sıra fazla ayırmak — Bölüm V'te çok işimize yarayacak.

Akıllı dizi: `std::vector`

Temel dizinin boyu derleme anında bellidir ve büyüyemez. **Akıllı dizi** (*vector*) bu sorunu çözer: gerekince kendi kendine büyür, boyunu bilir, kopyalanabilir.

akıllı dizi

```
#include <vector>
std::vector<int> d{3, 1, 4};
d.push_back(1);          // sona ekle
d.resize(10);            // on öğeye çıkar, yenileri sıfır
d.size();                // kaç öğe var
d[2];                    // erişim, sınır denetimi YOK
d.at(2);                 // erişim, sınır denetimi VAR (yavaş ama güvenli)

// iki boyutlu: 5 satır, her satırda 8 sıfır
std::vector<std::vector<int>> tablo(5, std::vector<int>(8, 0));
```

Hız kaybı var mı? Ölçtük: bir milyon öğelik dizide temel dizi 3 ms'de kuruluyor, akıllı dizi 7 ms'de. Sorgular ise ikisinde de mikrosaniyenin binde biri. Yani fark var ama sizin problemleriniz için **önemsiz**. Ölçmeyi Bölüm III'te öğreneceğiz — ve tahmin etmek yerine ölçmek, bu kitapçığın belki de en önemli alışkanlığı.

II.2

Eşlem: en çok işinize yarayacak kap

Eşlem (*map*) bir şeyi başka bir şeye bağlar: harften sayıya, addan telefona, şehirden komşularına. Dizide yer numarası 0, 1, 2 diye gitmek zorundadır; eşlemde "yer numarası" ne olursa olabilir. Ona

anahtar deniyor.

11-esleme.cpp — HARF SAYMA

```cpp
#include <iostream>
#include <map>
#include <string>
using namespace std;

using Sayac = map<char, int>;

int main() {
  string yazi{"kelebek kanadi"};

  Sayac kac;
  for (char h : yazi)
    if (h != ' ') ++kac[h];          // yoksa sıfırdan başlar

  for (auto [harf, adet] : kac)      // anahtarlar kendiliğinden sıralı
    cout << harf << " -> " << adet << endl;

  cout << "kaç ayrı harf: " << kac.size() << endl;

  // Dikkat! Olmayan bir anahtarı [] ile okumak onu YARATIR:
  cout << "z kaç kere: " << kac['z'] << endl;
  cout << "kaç ayrı harf: " << kac.size() << "  <- arttı!" << endl;

  // Doğrusu:
  cout << "q var mı: " << (kac.contains('q') ? "evet" : "hayır") << endl;
  cout << "kaç ayrı harf: " << kac.size() << "  <- değişmedi" << endl;
  return 0;
}
```

çıktı

```
a -> 2    b -> 1    d -> 1    e -> 3
i -> 1    k -> 3    l -> 1    n -> 1
kaç ayrı harf: 8
z kaç kere: 0
kaç ayrı harf: 9  <- arttı!
q var mı: hayır
kaç ayrı harf: 9  <- değişmedi
```

Üç şeye dikkat edin. Birincisi: `++kac[h]` satırı, harfi ilk kez görüyorsa sıfırdan başlatıp bir artırıyor — sayaç yazmanın en zarif yolu. İkincisi: eşlem anahtarları **kendiliğinden sıralı** tutuyor, siz hiçbir şey yapmadan çıktı alfabetik geliyor. Üçüncüsü de tuzak.

> **KLASİK TUZAK**
>
> Eşlemde **olmayan** bir anahtarı `[]` ile okursanız, C++ o anahtarı sessizce yaratır ve ilk değerini verir. Yani salt okuma sandığınız satır eşlemi büyütür. Sadece bakmak istiyorsanız `contains()` ya da `find()` kullanın. Bu hatayı derste yaptık ve bulmak yarım saatimizi aldı.

Küme, eşlemin küçük kardeşi

Küme (*set*) matematikteki kümenin ta kendisi: sıralı, tekrarsız. Aslında eşlemin değerini attığınızda geriye küme kalır. Tersi de doğru — her kümeyi bir eşlemle ifade edebilirsiniz:

küme ve eşlem

```
set<int> kume {1, 4, 6, 9};

// aynı bilgi, üstüne "kaç çarpanı var" bilgisi de eklenmiş hâliyle:
map<int, int> eslem {{1,1}, {4,3}, {6,4}, {9,3}};

// hatta çarpanların kendisi de saklanabilir:
map<int, set<int>> eslem2 {
    { 1, {1} },
    { 4, {1, 2, 4} },
    { 9, {1, 3, 9} }
};
```

Bu son yapıyı aklınızın bir köşesine yazın: “bir şeyden bir şey kümesine eşlem”. Bölüm V'te bir çizgeyi tam olarak böyle temsil edeceğiz: `map<Nokta, set<Nokta>>`, yani “her noktadan komşularına”.

II.3

Sıranın önemli olduğu kaplar

Üç özel kap daha var. Üçünün de yaptığı iş “öğe ekle, öğe çıkar” — aralarındaki tek fark **hangisinin çıkacağı**. Ama o tek fark, üçünü üç farklı algoritmanın kalbi yapıyor.

KAP	KURAL	YÖNTEMLER	NE İŞE YARAR
Yığın `std::stack`	Son giren ilk çıkar	`push` `pop` `top`	Derinlemesine gezi, parantez denetimi, geri alma
Kuyruk `std::queue`	İlk giren ilk çıkar	`push` `pop` `front`	Enlemesine gezi, en kısa yol (eşit adımlı)
Öncelik sırası `std::priority_queue`	En büyük önce çıkar	`push` `pop` `top`	Dijkstra, en yakın *n* şeyi bulmak

12-kaplar.cpp — AYNI GİRDİ, ÜÇ ÇIKTI

```cpp
#include <stack>
#include <queue>   // queue ve priority_queue ikisi de burada

stack<int> yigin;               // ilk giren son çıkar
queue<int> kuyruk;              // ilk giren ilk çıkar
priority_queue<int> oncelik;    // en büyük önce çıkar

for (int s : {3, 1, 4, 1, 5}) {
  yigin.push(s); kuyruk.push(s); oncelik.push(s);
}
```

çıktı

```
girdi   : 3 1 4 1 5
yığın   : 5 1 4 1 3   // ters
kuyruk  : 3 1 4 1 5   // aynı sırada
öncelik : 5 4 3 1 1   // büyükten küçüğe
```

Bu üç satır, ileride yazacağınız onlarca algoritmanın özeti. Aynı gezinme kodunda yığını kuyrukla değiştirdiğinizde derinlemesine arama enlemesine aramaya dönüşüyor — ve bambaşka bir soruyu çözüyor. Bölüm V'te bunu tam olarak yapacağız.

> **DENEYİN**
>
> `priority_queue` büyükten küçüğe sıralıyor. Küçükten büyüğe istiyorsanız üçüncü tür girdisine `std::greater<int>` verin: `priority_queue<int, vector<int>, greater<int>>`. Deneyin ve çıktının değiştiğini görün. Bu ayrıntı Dijkstra algoritmasında hayati olacak.

II.4

Kendi türünüzü yazmak

Şimdi işin en keyifli kısmı. Bir noktanın `x` ve `y`'sini iki ayrı değişkende taşımak yerine, ikisini bir araya getirip yeni bir tür yapabilirsiniz. Bunu yaptığınız an dil sizin problem alanınızın dilini konuşmaya başlar.

13-konum.cpp — İLK TÜRÜNÜZ

```cpp
struct Konum {
  int x{0}, y{0};                          // veriler

  void kaydir(int dx, int dy) {            // yöntem
    x += dx;
    y += dy;
  }
  double uzaklik() const {                 // const: hiçbir şeyi değiştirmem
    return sqrt(1.0 * x * x + y * y);
  }
};

// türün üyesi olmayan ama türü tanıyan bir işlemci:
ostream& operator<<(ostream& o, const Konum& k) {
  return o << "(" << k.x << ", " << k.y << ")";
}

int main() {
  Konum k{3, 4};
  cout << k << " başlangıca uzaklığı: " << k.uzaklik() << endl;

  k.kaydir(2, 2);
  cout << k << " şimdi uzaklığı: " << k.uzaklik() << endl;
}
```

```
(3, 4) başlangıca uzaklığı: 5
(5, 6) şimdi uzaklığı: 7.81025
```

`operator<<` satırına dikkat: `cout << k` yazabilmemizin sebebi bu. C++'ta işlemcilere yeni beceriler öğretebiliyorsunuz. Bir kere yazdınız mı, o türü artık her yerde `int` kadar rahat yazdırabiliyorsunuz. Hata ayıklarken bunun değeri paha biçilmez.

`struct` mu, `class` mı?

Teknik olarak tek fark: `struct`'ta üyeler varsayılan olarak **görünür** (`public`), `class`'ta **gizli** (`private`). Gelenek olarak ise `struct` "sadece veri torbası", `class` "kendi kurallarını koruyan bir nesne" için kullanılır.

bir sınıfın anatomisi

```
class Temel {
    int gizli_veri;          // varsayılan: private

protected:
    int korunmus_veri;       // alt türler görebilir

public:
    int gorunur_veri;

    Temel();                 // yapıcı: girdisi olabilir, çıktısı OLAMAZ
    Temel(const Temel&);     // kopyalayıcı yapıcı
    ~Temel();                // yıkıcı: girdisi de çıktısı da YOK, tek tane

    Temel& operator=(const Temel&);   // eşitleme

    friend ostream& operator<<(ostream&, const Temel&);
};
```

- **Yapıcı** (*constructor*) nesne doğarken çalışır: bellekte yer açar, ilk değerleri yazar. Birden fazla yapıcı olabilir.
- **Yıkıcı** (*destructor*) nesne ölürken çalışır ve tek tanedir. Açtığınız kaynakları burada kapatırsınız.
- `operator=` genelde `*this` döndürür — böylece `a = b = c` yazabilirsiniz. `this`, nesnenin kendi adresini tutan gösterge.
- `friend`, üye olmayan bir işleve gizli verileri görme izni verir. `operator<<` için tam da bu gerekir, çünkü o `ostream`'in soluna yazılır, sizin türünüzün değil.

Az sayıda değeri olan türler: `enum class`

Bir öğrencinin takımı 1 mi 2 mi? `int` kullanırsanız 7. takıma da yazabilirsiniz ve derleyici sesini çıkarmaz. Bunun yerine:

enum class

```
enum class Takim { Yok, Bir, İki };

std::vector<Takim> takim(n + 1, Takim::Yok);

using enum Takim;               // tür adını her seferinde yazmayalım
takim[b] = (takim[a] == Bir) ? İki : Bir;
```

Hem kod okunaklı oluyor hem de yanlışlıkla üçüncü bir takım kurmaya kalkarsanız derleyici hemen "olmaz" diyor. **Derleyicinin sizi yakalamasını sağlayan her tasarım kararı iyi bir karardır.**

II.5

Kalıplar: tür de bir değişken olabilir

Şimdiye kadar her değişkenin bir değeri, bir adı ve bir türü olduğunu söyledik. Şimdi tuhaf bir şey yapacağız: **türün kendisini değişken yapacağız.**

14-kalip.cpp — İŞLEV KALIBI

```cpp
template <typename Tur>
Tur ekle(Tur a, Tur b) { return a + b; }

int main() {
  cout << ekle(3, 4)                      << endl;   // Tur = int    -> 7
  cout << ekle(3.5, 4.25)                 << endl;   // Tur = double -> 7.75
  cout << ekle(string("ke"), string("lebek")) << endl;   // -> kelebek

  // cout << ekle("ke", "lebek");   // HATA! Neden?
}
```

`Tur` bir **tür değişkeni**. Değeri, kalıp kullanıldığında belli oluyor. Derleyici her çağrı için ayrı bir somut işlev üretiyor; buna *somutlaştırma* (*instantiation*) deniyor.

Son satır neden hata veriyor? Çünkü `"ke"` bir `std::string` değil, `const char*` yani bir **adres**. İki adresi toplamanın anlamı yok, derleyici de tam olarak bunu söylüyor:

derleyici — HATA

```
error: invalid operands of types 'const char*' and 'const char*'
       to binary 'operator+'
   |         return a + b;
```

Derleyici hatalarını okumayı öğrenin. Uzun ve korkutucu görünürler ama genelde ilk iki satırında her şey yazar. Bu alışkanlık sizi saatlerce zaman kazandırır.

Tür kalıpları

Aynı fikir türler için de geçerli — ve aslında bu bölümde kullandığımız `vector<int>`, `map<char,int>`, `set<int>` hepsi tür kalıbı:

tür kalıbı

```cpp
template <typename X>
class Kutu {
    X icerik;
public:
    Kutu(X x) : icerik{x} {}
    X al() const { return icerik; }
};

Kutu<int>    k1{42};
Kutu<Konum>  k2{{3, 4}};    // kendi türümüzle de çalışıyor!
```

Ders notlarımızda çok kullandığımız bir kolaylık da şu: uzun kalıp adlarına Türkçe takma ad vermek. Kod bir anda okunur hâle geliyor.

takma adlar

```
template <typename T> using Dizi = std::vector<T>;

using Dugum   = unsigned;
using Kume    = std::set<Dugum>;
using Cizge   = std::map<Dugum, Kume>;   // komşuluk kümesi
using Tahta   = Dizi<Dizi<bool>>;
```

II.6

Adres ve takma ad

Son bir kavram, ve belki de C++'ın en çok korkulan kavramı. Aslında çok basit: bir değişkenin **değeri** yerine bellekteki **yerini** taşımak.

15-adres.cpp — ÜÇ EKLEME, ÜÇ SONUÇ

```
using Sayi  = int;
using Adres = Sayi*;      // Sayı türünün adresi de bir tür

void ekleA(Adres a, Sayi ek) { *a += ek; }   // adresle
void ekleT(Sayi& t, Sayi ek) { t  += ek; }   // takma adla
void ekleK(Sayi  k, Sayi ek) { k  += ek; }   // kopyayla (etkisiz!)

int main() {
  Sayi s{10};

  ekleK(s, 10);  cout << "kopyadan sonra   : " << s << endl;
  ekleA(&s, 10); cout << "adresten sonra   : " << s << endl;
  ekleT(s, 10);  cout << "takma addan sonra: " << s << endl;
}
```

çıktı

```
kopyadan sonra   : 10   // hiçbir şey olmadı!
adresten sonra   : 20
takma addan sonra: 30
```

Üç işlev de “ekle” diyor ama biri hiçbir işe yaramıyor. `ekleK` değişkenin *ikizini* alıyor, ikizi artırıyor, sonra ikiz ölüyor. `ekleA` adresi alıyor, `*` ile o adrestekine uzanıyor. `ekleT` ise sadece tek bir `&` imgesiyle değişkenin kendisine **takma ad** (*reference*) veriyor.

C dilinde sadece adresler vardı; takma ad C++'la geldi. Modern kodda tercih genelde takma ad: daha sade ve daha güvenli, çünkü takma ad hiçbir zaman boş olamaz. Ama akıllı dizi yerine temel dizi kullandığınızda ya da `new` ile bellek ayırdığınızda adreslere geri dönersiniz.

PÜF NOKTASI

Büyük bir kabı işleve girdi olarak verirken `const Dizi&` yazın. `const` "değiştirmeyeceğim" sözü, `&` ise "kopyalamayın" demek. Bir milyon öğelik bir diziyi her çağrıda kopyalamak, çözümünüzün neden yavaş olduğunu anlayamadığınız durumların en yaygın nedenidir.

ALIŞTIRMALAR

1. **Sözlük.** `map<string, vector<string>>` kullanarak çok tanımlı bir sözlük yapın: bir sözcüğün birden fazla anlamı olabilsin. Sonra sözcük ekleme ve arama işlevleri yazın.

2. **Dengeli parantez.** Bir yazıda parantezler doğru açılıp kapanıyor mu? Önce sadece `(` ve `)` için çözün — şaşırtıcı biçimde tek bir sayaçla olur, yığına gerek yok. Sonra `[]` ve `{}` ekleyin. `ŞİMDİ YIĞIN GEREKLİ` Neden gerekli oldu, onu da anlatabilir misiniz?

3. **Tekrar var mı?** Bir sayı dizisinde tekrar eden öğe var mı? En az üç farklı yöntem bulun ve hızlarını karşılaştırın. (Biri `set` kullanır, biri sıralama, biri iç içe iki döngü.)

4. **Kendi `Kesir` türünüz.** Pay ve paydası olan bir tür yazın. `+`, `*` ve `<<` işlemcilerini tanımlayın. Yapıcı içinde kesri sadeleştirin (`3/6` yerine `1/2`). `BONUS` Payda sıfır verilirse ne yapmalı?

5. **En büyük ve en küçük.** Bir dizinin hem en büyük hem en küçük öğesini, **en az sayıda karşılaştırmayla** bulun. Karşılaştırmaları programa saydırın. Öğeleri ikişer ikişer ele almayı deneyin; n öğe için `2n-2`'den az yapabilir misiniz?

BÖLÜM III

Programcının Zanaatı

Bu bölümde tek bir yeni C++ kavramı yok. Onun yerine, iyi programcıyı acemi programcıdan ayıran şey var: kodu derlemek, sınamak, hatayı bulmak ve hızı tahmin etmek yerine ölçmek. Bu alışkanlıkları edinen bir öğrenci, dili daha iyi bilen ama bunları bilmeyen bir öğrenciyi kolayca geçer.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

III.1

Tarayıcıdan terminale

Çevrimiçi derleyiciler harika bir başlangıç. Ama bir noktada kendi bilgisayarınıza inmelisiniz: büyük testler tarayıcıda çalışmaz, hata ayıklayıcı orada zayıftır ve gerçek projeler tek dosyaya sığmaz.

terminal — BİLMENİZ GEREKEN SEKİZ KOMUT

```
pwd                   # neredeyim?
ls                    # burada ne var?
ls -l                 # ayrıntılı liste
cd ileri/dersler      # dizine gir
cd ..                 # bir üste çık
mkdir yeni-alistirma  # dizin aç
cat dosya.cpp         # dosyayı ekrana bas
./prog < test1        # programa dosyadan girdi ver
```

Son satır yarışmalarda hayat kurtarır: girdiyi her seferinde elle yazmak yerine bir dosyaya koyup `<` ile beslersiniz.

Derleyici tek bir program değil

`c++ ana.cpp -o prog` yazdığınızda aslında iki iş art arda yapılıyor: önce **derleme** (kaynak koddan makine koduna), sonra **bağlama** (parçaları tek çalıştırılabilir dosyada birleştirme). İkisini ayırabilirsiniz:

terminal — DERLEME VE BAĞLAMA

```
# tek adımda
$ c++ -std=c++23 ana.cpp asal.cpp -o prog

# ya da iki adımda: önce nesne kodu (.o), sonra bağlama
$ c++ -std=c++23 -c ana.cpp      # -> ana.o
$ c++ -std=c++23 -c asal.cpp     # -> asal.o
$ c++ -std=c++23 ana.o asal.o -o prog

# faydalı seçenekler
$ c++ -std=c++23 -Wall -Wextra ana.cpp -o prog   # bütün uyarıları göster
$ c++ -std=c++23 -g ana.cpp -o prog              # hata ayıklama bilgisi ekle
$ c++ -std=c++23 -O2 ana.cpp -o prog             # iyileştirmeleri aç (hızlı!)
```

Neden ayıralım? Çünkü yirmi dosyalık bir projede tek bir dosyayı değiştirdiğinizde diğer on dokuzunu yeniden derlemek anlamsız. İşte `make` tam olarak bunun için var.

PÜF NOKTASI

`-Wall` seçeneğini **her zaman** açık tutun. Derleyici uyarıları can sıkıcı değil, bedava kod incelemesidir. Derste bir işlevin çıktı türünü yanlış yazmıştık; program tesadüfen doğru çalışıyordu ama derleyici bunu `-Wall` ile baştan söylemişti. Uyarıyı okusaydık yarım saat kazanacaktık.

III.2

`make`: tek komutla her şey

`make`, bir dosyanın bağlı olduğu dosyalardan daha eski olup olmadığına bakar ve sadece gerekeni yeniden yapar. Kuralları `Makefile` adlı dosyaya yazarsınız. Biçim çok basit:

Makefile biçimi

```
<hedef>: <bağlı olduğu şeyler>
<SEKME>çalıştırılacak komut
```

EN SİNİR BOZUCU HATA

Komut satırının başındaki girinti **sekme** (Tab) olmak zorunda, boşluk değil. Editörünüz sekmeyi boşluğa çeviriyorsa `make` anlaşılmaz bir hata verir. `Makefile` düzenlerken bu dönüşümü kapatın.

Şimdi gerçek bir örnek. Asal çarpan bulan, üç dosyaya bölünmüş, kendi kendini sınayan küçük bir proje:

Makefile — ASAL-PROJESİ

```
hedef: derle çalıştır

derle: prog

prog: ana.o asal.o
    c++ -std=c++23 ana.o asal.o -o prog

ana.o: ana.cpp asal.h
    c++ -std=c++23 -c ana.cpp

asal.o: asal.cpp asal.h
    c++ -std=c++23 -c asal.cpp

çalıştır: prog
    ./prog 60

test: prog
    ./prog 360 > test-ciktisi
    diff beklenen-cikti test-ciktisi
    rm -f test-ciktisi
    @echo "TEST GEÇTİ"

temizle:
    rm -f prog *.o test-ciktisi
```

Hedef adlarının Türkçe olabildiğine dikkat edin — `make derle`, `make test`, `make temizle`. Bir de her hedefin **neye bağlı olduğunu** doğru yazmak önemli: `ana.o` hem `ana.cpp`'ye hem `asal.h`'ye bağlı, çünkü başlık dosyası değişirse yeniden derlenmesi gerek.

terminal — ÇALIŞIRKEN

```
$ make
c++ -std=c++23 -c ana.cpp
c++ -std=c++23 -c asal.cpp
c++ -std=c++23 ana.o asal.o -o prog
./prog 60
denemeler geçti.
60 sayısının asal çarpanları: 2 2 3 5

$ make          # ikinci kez: hiçbir şey değişmedi
make: 'hedef' hedefi için yapılacak bir şey yok.
```

TAKIM İŞİ

Yeni bir alıştırmaya başlarken hazır bir düzenden kopyalayın: bir `Makefile`, bir `ana.cpp`, bir de `beklenen-cikti`. Ders deposundaki `sablon/` dizini tam olarak bunun için var. Takımca bir kere kurun, dönem boyunca kopyalayın — her seferinde sıfırdan Makefile yazmak zaman kaybı.

Kendi kendini sınayan program

Bir program yazdınız, çalıştırdınız, doğru yanıt verdi. Sonra bir yerini değiştirdiniz. Hâlâ doğru mu? Bunu her seferinde elle denemek imkânsız. Çözüm: **sınamayı programın içine koymak**.

En sade araç `<cassert>` içindeki `assert`. İçindeki koşul yanlışsa program hemen durur ve hangi satırda durduğunu söyler.

ana.cpp — DENEMELER

```
#include <cassert>
#include "asal.h"

void dene() {
  assert(asalMi(2));
  assert(asalMi(97));
  assert(not asalMi(1));
  assert(not asalMi(91));                // 7 * 13, kolay tuzak
  assert(carpanlar(60).size() == 4);     // 2, 2, 3, 5
  std::cout << "denemeler geçti." << std::endl;
}

int main(int argc, char** argv) {
  dene();                                    // her çalıştırışta sınansın
  int n = (argc > 1) ? std::stoi(argv[1]) : 60;
  // ...
}
```

`assert(not asalMi(91))` satırına özellikle dikkat. İyi bir sınama, **doğru olması gerekeni değil, yanlış olması gerekeni** de sınar. 91 sayısı asal görünür ama `7 × 13`'tür; naif bir asal denetimi tam burada patlar.

Şimdi kasten bir hata yapalım: çarpan bulan işlevin son satırını silelim. Ne olduğuna bakın:

terminal — YAKALANDI

```
$ make test
c++ -std=c++23 -c asal.cpp
c++ -std=c++23 ana.o asal.o -o prog
./prog 360 > test-ciktisi
prog: ana.cpp:10: void dene(): Assertion `carpanlar(60).size() == 4' failed.
Aborted
make: *** [Makefile:18: test] Error 134
```

Hata dosya ve satır numarasıyla birlikte geldi. Bu, üç saatlik bir aramayı üç saniyeye indiren şey.

Altın dosya yöntemi

`assert` tek tek değerleri sınar. Peki programın **bütün çıktısını** nasıl sınarız? Ders deposunda kullandığımız yöntem çok basit ve çok etkili: doğru çıktıyı bir kere üretip bir dosyaya kaydedin (buna **altın dosya** deniyor), sonra her seferinde `diff` ile karşılaştırın.

Makefile — TEST HEDEFİ

```
test: prog
    ./prog 360 > test-ciktisi
    diff beklenen-cikti test-ciktisi
    rm -f test-ciktisi
```

Çıktıda tek bir boşluk bile kaysa `diff` haber veriyor:

terminal — DİFF YAKALADI

```
$ make test
./prog 360 > test-ciktisi
diff beklenen-cikti test-ciktisi
2c2
< 360 sayısının asal çarpanları: 2 2 2 3 3 5
---
> 360  sayısının asal çarpanlari: 2 2 2 3 3 5
make: *** [Makefile:19: test] Error 1
```

DİKKAT

`diff` fark gösterdiğinde iki ihtimal var: ya kodunuzu bozdunuz, ya da çıktıyı **bilerek** değiştirdiniz. İkincisiyse altın dosyayı güncelleyin. Ama önce farka gerçekten bakın — altın dosyayı düşünmeden güncellemek, testi tamamen silmekle aynı şey.

Bir de zamana bağlı çıktılara dikkat: “doğduğunuzdan beri kaç gün geçti” yazan bir programın altın dosyası her gün bozulur. Böyle çıktıları ya sabitleyin ya da testin dışında tutun.

III.4

Hata bulmanın üç yolu

1. Yazdırarak — ama açılıp kapanabilir biçimde

En yaygın hata ayıklama yöntemi programın içine `cout` serpiştirmek. Sorun şu: iş bitince hepsini tek tek silmek gerekiyor, sonra hata tekrarlayınca hepsini geri yazmak. Şu küçük numara bu derdi bitiriyor:

dout numarası

```
bool yaz = false;              // yazsın istiyorsak true yapalım
ostream nout{nullptr};          // hiçbir yere yazmayan akım
ostream& dout = yaz ? cout : nout;

dout << "k=" << k << " toplam=" << toplam << endl;
```

Artık bütün ayıklama çıktılarınızı `dout` ile yazın. Tek bir `bool` değiştirerek hepsini birden açıp kapatabilirsiniz. (*debug out*'un kısaltması.)

2. Hata ayıklayıcıyla

gdb, C ve C++ dünyasının efsanevi hata ayıklayıcısı. Programı satır satır çalıştırıp her adımda değişkenlerin içine bakmanızı sağlar. Önce `-g` ile derleyin:

terminal GDB

```
$ c++ -std=c++23 -g ana.cpp -o prog
$ gdb ./prog

(gdb) break main          # main'de dur
(gdb) run                 # başlat
(gdb) next                # bir satır ilerle (işlevin içine girme)
(gdb) step                # bir satır ilerle (işlevin içine gir)
(gdb) print toplam        # değişkenin değerini göster
(gdb) backtrace           # buraya hangi çağrılarla geldik?
(gdb) continue            # bir sonraki durağa kadar devam
(gdb) quit
```

OnlineGDB'de aynı şey tuşlarla yapılıyor: satır numarasına tıklayıp durak koyun, **Debug**'a basın, sonra **step into** ile ilerleyin. Özyineleyen bir işlevin nasıl çalıştığını anlamanın en hızlı yolu budur — Bölüm IV'te işinize yarayacak.

3. Küçülterek

Belki de en güçlü yöntem, ve hiçbir araç gerektirmiyor. Kod büyük ve hata nerede belli değilse, **hatayı yeniden üreten en küçük örneği** bulun. Girdiyi yarıya indirin: hâlâ hata veriyor mu? Yarısını daha indirin. Beş adımda bin satırlık girdi otuz satıra iner ve hata çoğu zaman kendini gösterir.

Ders deposundaki her ciddi hata bu şekilde bulundu. Bir keresinde bir düğüm kuyruğa iki kere giriyordu; yedi düğümlük minik bir çizgede çıktıyı elle okuyunca hemen göründü. Yüz düğümlük çizgede asla göremezdik.

> **TAKIM İŞİ**
>
> Hata avında ikili çalışmanın faydası en yüksek. Bir kişi kodu yüksek sesle okur, öteki “dur, burada ne olur?” diye sorar. İnsan kendi kodunda ne yazdığını değil, *ne yazmak istediğini* okur; ikinci bir çift göz bu körlüğü kırar.

Tahmin etmeyin, ölçün

"Bu kod yavaş galiba" cümlesi bir bilgi değil, bir his. C++ size hissi ölçüye çevirecek bir araç veriyor: `<chrono>`.

20-hiz.cpp — SÜRE ÖLÇÜMÜ

```cpp
#include <chrono>
using saat = std::chrono::high_resolution_clock;

auto basla = saat::now();

const int n = 1'000'000;              // tırnaklar okumayı kolaylaştırır
std::vector<int> dizi(n);
for (int k = 0; k < n; ++k) dizi[k] = 2 * k;

auto bitti = saat::now();
auto insa = std::chrono::duration_cast<std::chrono::milliseconds>(bitti - basla);

std::cout << "inşa süresi: " << insa.count() << " ms" << std::endl;
```

Bu programı iki kere derleyip çalıştırdım. Tek fark derleyici seçeneği:

DERLEME	BİR MİLYON ÖĞE İNŞASI	ORTALAMA ERİŞİM
`-O0` (iyileştirme yok)	8 ms	2.66 ns
`-O2` (iyileştirme açık)	2 ms	0.28 ns

Aynı kod, dört kat hızlı inşa, on kat hızlı erişim. Tek satır kod değiştirmeden. Yarışmalarda bu fark, "zaman aşımı" ile "kabul edildi" arasındaki fark olabilir.

Terminalde bütün programın süresini ölçmenin daha da kolay bir yolu var:

terminal — TİME

```
$ time ./prog < buyuk-test
...
./prog < buyuk-test  28.71s user 0.11s system 97% cpu 29.430 total
```

Bu, derste gerçekten karşımıza çıkan bir ölçüm: labirentten kaçış problemini çözen ilk programımız yarım dakika sürüyordu. Algoritmayı düzeltince aynı girdide 0.73 saniyeye indi — kırk kat. Nasıl olduğunu Bölüm V'te göreceksiniz. Şimdilik şunu aklınızda tutun: **büyük hız farkları derleyici seçeneğinden değil, algoritmadan gelir.**

Kendini tekrar etme

Yazılımın belki de en meşhur ilkesi: **DRY** (*Don't Repeat Yourself*). Aynı kodu kopyalayıp yapıştırdığınızda, onu bir kere değil *iki kere* bakıma almış olursunuz. Bir hata bulup birini düzeltir, ötekini unutursunuz.

Derste yedi kere iç içe `for` döngüsü içeren, dört farklı yerde neredeyse aynı biçimde tekrarlanan bir çizim koduna baktık. Çalışıyordu, hem de güzel çalışıyordu. Ama o dört bloğu tek bir *işlev kalıbına* çevirince kod hem kısaldı hem de yeni çizim eklemek dakikalar sürer oldu.

Sadeleştirme (*refactoring*)

Kodu, davranışını değiştirmeden daha anlaşılır hâle getirmeye **sadeleştirme** deniyor. Öğrenciyken bunun boş bir uğraş gibi görünmesi çok normal — "çalışıyor ya, ne gerek var?" Gerek var, çünkü kodu bir kere yazarsınız, on kere okursunuz.

İki gerçek örnek, ders notlarından:

PROGRAM	ÖNCE	SONRA	NE KAZANDIK
Labirentte en kısa yol	93 satır	69 satır	Aynı puan, çok daha okunur kod
Canavarlardan kaçış	141 satır	100 satır	Hem kısaldı hem **40 kat hızlandı**

İkinci satır çok öğretici: uzun program yavaş, kısa program hızlıydı. Bu tesadüf değil. Kodu sadeleştirirken ne yaptığınızı daha net görürsünüz, ve o netlikte daha iyi algoritma saklıdır.

Sadeleştirme için üç somut hamle

- **İyi ad koyun.** `int a, b, c;` yerine `int satir, sutun, uzaklik;`. Bir değişkenin adını doğru koyduğunuzda kodun yarısı kendini yazar.
- **Tür takma adları kullanın.** `vector<vector<pair<int,int>>>` yerine `using Cizge = Dizi<Baglar>;`. Kod problemin dilinde konuşmaya başlar.
- **Tekrarlayan bloğu işlev yapın.** Aynı beş satırı ikinci kez yazdığınızı fark ettiğiniz an durun ve ona bir ad verin.

ALIŞTIRMALAR

1. **Kendi şablonunuzu kurun.** Bir dizin açın; içine `Makefile`, `ana.cpp` ve `beklenen-cikti` koyun. `make`, `make test` ve `make temizle` çalışsın. Bundan sonraki her alıştırmada bunu kopyalayın.

2. **Testi kırın.** Çalışan bir programınızın çıktısını tek bir karakter değiştirin ve `make test`'in nasıl bir hata verdiğini görün. Sonra geri alın. Testin gerçekten çalıştığını görmemiş olsaydınız, ona güvenemezdiniz.

3. **Ölçün.** Bölüm II'deki "tekrar var mı" alıştırmasının üç çözümünü `<chrono>` ile ölçün. On bin, yüz bin ve bir milyon öğeyle deneyin. Süreler nasıl artıyor? Bir tablo yapın. ÖNEMLİ Bu tablo, bir sonraki bölümde konuşacağımız *karmaşıklık* kavramının somut hâli.

4. **gdb ile özyineleme izleyin.** Fibonaççi'yi özyinelemeli yazın, `fib(5)` için `backtrace` komutunu birkaç kere çalıştırın. Çağrı yığınının büyüyüp küçüldüğünü kendi gözünüzle görün.

5. **Bir kodu sadeleştirin.** İki hafta önce yazdığınız bir programı açın. Anlamakta zorlandığınız her satır bir fırsat: değişkeni yeniden adlandırın, tekrarı işleve çıkarın, sihirli sayıya isim verin. Sonunda `make test` hâlâ geçiyor mu? Geçiyorsa doğru sadeleştirdiniz.

BÖLÜM IV

Özyineleme ve Arama

Bir işlev kendi kendini çağırabilir. Kulağa hile gibi geliyor, ama bu tek fikir Hanoi kulelerinden sekiz vezire, altküme üretiminden bütün bir çizge kuramına kadar uzanıyor. Bu bölümde onu ehlileştireceğiz — ve bir ara kırk bin kat hızlanacağız.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

IV.1

Kendini çağıran işlev

Alışık olduğumuz işlev tanımı şöyle: `y = f(x)`. Girdi ver, çıktı al. **Özyinelemeli** (*recursive*) tanım ise bir şeyi *kendi küçük hâliyle* anlatır:

iki tür tanım

```
// klasik tanım: her x için bir y
y = f(x)

// özyinelemeli tanım
f(0) = sabit                              // TABAN
f(n) = g( f(n-1), f(n-2), ..., f(0) )     // ADIM
```

Her özyinelemeli işlevin iki parçası var ve **ikisi de olmazsa olmaz**:

1. **Taban durumu.** Kendini çağırmadan yanıt verdiği durum. En küçük, en kolay hâl.
2. **Adım.** Problemi kendinden *daha küçük* bir problemle ifade etmesi. Küçülmesi şart, yoksa hiç durmaz.

> **BİR NUMARALI HATA**
>
> Taban durumunu unutmak ya da yanlış yazmak. Program ya sonsuza kadar çalışır ya da bir süre sonra çöker (“*stack overflow*”: çağrı yığını taşar). Derste tam olarak bunu yaptık: bir çizge gezisinde “daha önce buraya uğradık mı” koşulunu yazmayı unuttuk ve program sonsuz döngüye girdi. Özyinelemeli bir işlev yazarken **ilk yazacağınız satır taban durumu olsun.**

Hanoi kuleleri

Özyinelemenin gücünü en iyi gösteren bulmaca. Üç çubuk var, birincisinde büyükten küçüğe dizilmiş *n* disk. Hepsini üçüncü çubuğa taşıyacaksınız. Kurallar: her seferinde tek disk, ve büyük disk asla küçüğün üstüne konmayacak.

Döngüyle çözmeye kalkarsanız saatlerinizi alır. Özyinelemeyle bakış açısı şu: *en büyük diski hedefe koyabilmem için, üstündeki n−1 diskin yoldan çekilmesi gerek.* O da aynı problemin küçüğü!

34-hanoi.cpp — ÜÇ SATIRLIK ÇÖZÜM

```
void tasi(int n, int kaynak, int hedef, int yardimci) {
  if (n == 0) return;                       // TABAN: taşınacak disk yok

  tasi(n-1, kaynak, yardimci, hedef);       // üstteki n-1 diski aradaki çubuğa
  cout << ++adim << ") " << n << ". diski "
       << kaynak << " -> " << hedef << endl;   // en büyüğü hedefe
  tasi(n-1, yardimci, hedef, kaynak);       // n-1 diski üstüne
}

// çağırırken: tasi(3, 1, 3, 2);
```

çıktı — N = 3

```
1) 1. diski 1 -> 3
2) 2. diski 1 -> 2
3) 1. diski 3 -> 2
4) 3. diski 1 -> 3
5) 1. diski 2 -> 1
6) 2. diski 2 -> 3
7) 1. diski 1 -> 3
toplam 7 adım (2^3 - 1 = 7)
```

Üç satırlık bir işlev, mükemmel bir çözüm üretiyor. Adım sayısı da $2^n - 1$. Efsaneye göre 64 diskli bir kule tamamlandığında dünya sona erecekmiş; saniyede bir hamleyle bu yaklaşık **584 milyar yıl** sürer. Rahat olun.

> **DENEYİN**
>
> `n`'i 4, 5, 6 yapın ve adım sayısını not edin. Sonra `tasi()` içindeki üç satırın sırasını değiştirin. Çıktı hâlâ geçerli bir çözüm mü? Neden değil?

IV.2

Aynı işi iki kere yapmayın

Özyinelemenin bir tuzağı var. Fibonaççi'yi tanımıyla yazalım:

tanımı yazdık, oldu

```
Sayi fibYavas(int n) {
  if (n < 2) return n;
  return fibYavas(n-1) + fibYavas(n-2);
}
```

Doğru çalışıyor. Ama `fib(40)` deneyin, çayınızı içecek vakit bulursunuz. Neden? Çünkü sağ tarafta **iki** çağrı var; her çağrı ikiye çatallanıyor, o ikisi dörde... Ağacın altında `fib(3)` milyonlarca kere yeniden hesaplanıyor.

Çare çok basit: **hesapladığını bir kenara yaz**. Buna **bellekle hızlandırma** (*memoizasyon*) deniyor.

30-fib.cpp — TEK SATIR FARK

```
map<int, Sayi> bellek;

Sayi fibHizli(int n) {
  if (n < 2) return n;
  if (bellek.contains(n)) return bellek[n];      // <- eklenen satır
  return bellek[n] = fibHizli(n-1) + fibHizli(n-2);
}
```

İki işlevi `n = 40` için çalıştırıp çağrıları saydırdım. Sonuç şaşırtıcı:

İŞLEV	ÇAĞRI SAYISI	SÜRE
`fibYavas(40)`	331 160 281	321 613 µs
`fibHizli(40)`	79	8 µs

Üç yüz otuz bir milyon çağrı yerine yetmiş dokuz. Kırk bin kat hız — **tek bir satırla**. Kodun mantığı değişmedi, sadece unutkanlığı düzeldi.

Bu fikrin bir adı var ve bir bölümü hak ediyor: dinamik programlama. Bölüm VI'da oraya döneceğiz.

> **TAKIM İŞİ**
>
> Biriniz `fibYavas(35)` çalıştırırken, öteki `fib(35)`'i kâğıt üzerinde tanımdan hesaplamaya başlasın. Kim önce bitirir dersiniz? Sonra `fibHizli` ile yarışın. Bu küçük oyun, "algoritma seçimi"nin ne demek olduğunu bir ömür anlatır.

Bütün olasılıkları gezmek

Şimdi özyinelemenin asıl gücüne geliyoruz. Çoğu zor problemde "bütün olasılıkları dene"den başka yol yoktur. Buna **geri dönüşlü arama** (*backtracking*) deniyor: bir seçim yap, o dalda ilerle, çıkmaza girersen **geri dön**, başka bir seçim dene.

Kalıbı her seferinde aynı ve ezberlemeye değer:

geri dönüşlü arama kalıbı

```
void ara(durum) {
  if (tamamlandı) { kaydet(); return; }

  for (auto secim : olasiSecimler) {
    if (not uygun(secim)) continue;

    uygula(secim);          // 1. seç
    ara(yeniDurum);         // 2. o dalda derine in
    geriAl(secim);          // 3. BIRAKTIĞIN GİBİ BIRAK
  }
}
```

Üçüncü satır en çok unutulanı. Geri almayı unutursanız, bir daldaki seçiminiz diğer dalları kirletir ve program sessizce yanlış yanıt verir.

Bütün altkümeler

`{3, 1, 2}` kümesinin bütün altkümelerini üretelim. Her öğe için tek bir karar var: *alayım mı, almayayım mı?*

31-altkume.cpp — AL YA DA ALMA

```
Dizi kume{3, 1, 2}, secilen;

void uret(size_t k) {
  if (k == kume.size()) { yaz(); return; }   // TABAN: karar bitti

  uret(k + 1);                    // k'ıncı öğeyi ALMA

  secilen.push_back(kume[k]);
  uret(k + 1);                    // k'ıncı öğeyi AL
  secilen.pop_back();             // geri dön: bıraktığın gibi bırak
}
```

Aynı işi **hiç özyineleme kullanmadan**, bitlerle de yapabiliriz. n öğe için 2^n altküme var; her sayının ikili yazılışı bir altkümeyi tarif ediyor:

çıktı

```
sayı  bitler  altküme
----  ------  --------
 0     000    { }
 1     001    { 3 }
 2     010    { 1 }
 3     011    { 3 1 }
 4     100    { 2 }
 5     101    { 3 2 }
 6     110    { 1 2 }
 7     111    { 3 1 2 }
```

bitlerle üretim

```
size_t n = kume.size(), altKumeSayisi = 1u << n;   // 2^n

for (size_t ak = 0; ak < altKumeSayisi; ++ak)
  for (size_t k = 0; k < n; ++k)
    if (ak & (1u << k))         // k'ıncı bit açık mı?
      cout << kume[k] << " ";
```

İkisini de bilmekte fayda var. Bit yöntemi daha hızlı ve bellek yemiyor, ama sadece *n* 60'tan küçükken kullanılabiliyor. Özyineleme ise her boyutta çalışıyor ve **budama** yapmaya izin veriyor — birazdan göreceğimiz gibi bu çok kritik.

Bütün permütasyonlar

Aynı kalıp, tek fark: bu sefer *sıra* önemli, ve her öğeyi bir kere kullanabiliriz. Onun için "bunu daha önce kullandım mı" diye bir dizi tutuyoruz.

32-permutasyon.cpp — N = 3

```
vector<int> perm;
vector<bool> eklendi(N, false);

void uret(int derinlik) {
  if ((int) perm.size() == N) { yaz(perm); return; }

  for (int k = 0; k < N; ++k) {
    if (eklendi[k]) continue;        // bunu zaten kullandık

    eklendi[k] = true;
    perm.push_back(k);
    uret(derinlik + 1);
    perm.pop_back();               // geri al
    eklendi[k] = false;           // geri al
  }
}
```

Bu programa bir izleme satırı ekleyip çalıştırdım. Çıktıyı okuyun — özyinelemenin nasıl derine inip geri döndüğünü *görebilirsiniz*. `+` işaretleri derinliği gösteriyor:

izleme — ÖZYİNELEME CANLI

```
+ k=0  perm: 0
++ k=1  perm: 01
+++ k=2  perm: 012
       1. permütasyon: 012
++ k=2  perm: 02
+++ k=1  perm: 021
       2. permütasyon: 021
+ k=1  perm: 1
++ k=0  perm: 10
+++ k=2  perm: 102
       3. permütasyon: 102
++ k=2  perm: 12
+++ k=0  perm: 120
       4. permütasyon: 120
+ k=2  perm: 2
++ k=0  perm: 20
+++ k=1  perm: 201
       5. permütasyon: 201
++ k=1  perm: 21
+++ k=0  perm: 210
       6. permütasyon: 210
```

PÜF NOKTASI

Standart kütüpte hazırı var: `std::next_permutation(d.begin(), d.end())`. Diziyi bir sonraki permütasyona çevirir, bitince `false` döner. Bir `do…while` döngüsüyle hepsini üretirsiniz. Ama önce kendiniz yazın — hazırı kullanmak, nasıl çalıştığını bilerek kullanmakla aynı şey değil.

IV.4

Sekiz vezir

Klasiklerin klasiği. Satranç tahtasına sekiz vezir yerleştirin; hiçbiri diğerini tehdit etmesin. Vezir yatay, dikey ve çapraz gider — yani her satırda tam bir vezir olmak zorunda. Bu gözlem problemi yarı yarıya küçültüyor: artık soru “her satırdaki vezir hangi sütunda?”

Püf noktası köşegenleri numaralandırmakta. Sol-üstten sağ-alta giden bir köşegen üzerinde `satır + sütun` hep aynıdır; öteki yönde ise `satır − sütun` aynıdır. Böylece “bu köşegen dolu mu?” sorusu tek bir dizi bakışına iniyor.

33-vezir.cpp — TAM ÇÖZÜM

```cpp
const int N = 8;
vector<bool> sutun(N, false),          // bu sütunda vezir var mı?
             capraz1(2*N-1, false),    // satır + sütun sabit
             capraz2(2*N-1, false);    // satır - sütun sabit
int cozum = 0;

void koy(int satir) {
  if (satir == N) { ++cozum; return; }        // sekiz vezir yerleşti

  for (int s = 0; s < N; ++s) {
    int c1 = satir + s, c2 = satir - s + N - 1;
    if (sutun[s] or capraz1[c1] or capraz2[c2]) continue;   // tehdit altında

    sutun[s] = capraz1[c1] = capraz2[c2] = true;   // vezir kondu
    koy(satir + 1);                                // bir alt satıra geç
    sutun[s] = capraz1[c1] = capraz2[c2] = false;  // geri al
  }
}
```

Kalıbı tanıdınız mı? Altküme ve permütasyon üretimiyle **birebir aynı**: seç, derine in, geri al. Değişen sadece “uygun mu?” koşulu.

Programı farklı tahta boylarında çalıştırdım. Sonuçlar hiç de düzenli değil:

TAHTA	4×4	5×5	6×6	7×7	8×8
Çözüm sayısı	2	10	4	40	92

6×6 tahtada 5×5'ten *daha az* çözüm var. Neden? Kimse basit bir formül bulamadı; bu diziyi hesaplamanın bilinen tek yolu saymak. Matematiğin hoşuma giden yanı da bu: bazen en sade sorunun bile kısa yolu yok.

IV.5

Ne kadar sürer? Karmaşıklık

“Bütün olasılıkları dene” iyi bir fikir, ta ki olasılıklar tükenmez olana kadar. Bir algoritmanın girdi büyüdükçe ne kadar yavaşladığını anlatan gösterime **büyük O** deniyor. Sabitleri ve küçük terimleri atarız; çünkü mesele bir milisaniye değil, *büyüme biçimi*.

KARMAŞIKLIK	ADI	N=20	N=1000	$N=10^6$	ÖRNEK
`O(1)`	sabit	1	1	1	Dizide öğeye erişim
`O(log n)`	logaritmik	4	10	20	İkili arama, tahmin oyunu
`O(n)`	doğrusal	20	10^3	10^6	Diziyi bir kere tarama
`O(n log n)`	—	86	10^4	2×10^7	Sıralama, Dijkstra
`O(n²)`	karesel	400	10^6	10^{12}	İç içe iki döngü
`O(2ⁿ)`	üstel	10^6	∞	∞	Bütün altkümeler
`O(n!)`	faktöriyel	10^{18}	∞	∞	Bütün permütasyonlar

Tabloyu bir kere gerçekten okuyun. `O(n²)` bir milyon öğe için `10¹²` işlem demek — saatler. `O(2ⁿ)` ise *n* kırkı geçince evrenin yaşını aşıyor. Bu yüzden yarışma sorularında girdi sınırına bakmak, algoritma seçmenin ilk adımıdır:

> **KABA KURAL**
>
> Bilgisayarınız saniyede kabaca **10^8** basit işlem yapar. Soruda `n ≤ 20` yazıyorsa üstel çözüm serbest. `n ≤ 5000` ise karesel olabilir. `n ≤ 10⁶` ise `O(n)` ya da `O(n log n)` aramanız gerek. Bu tek cümle, boşa harcanacak saatlerinizi kurtarır.

Arama uzayını budamak

Üstel bir arama uzayıyla karşılaştığınızda ne yapılır? Bütünüyle kurtulamazsınız, ama **meyve vermeyecek dalları kesebilirsiniz**. Buna **budama** (*pruning*) deniyor.

Derste ızgara üzerinde bütün *Hamilton patikalarını* (her kareden tam bir kere geçen yollar) arayan bir program yazdık. 6×6 tahtada saniyeler sürüyordu, 7×7'de dakikalar. Sonra şu iki satırı ekledik:

budama — DAKİKALAR → SANİYENİN ALTI

```
bool so = gir[y][x-1], sa = gir[y][x+1],
     yu = gir[y-1][x], as = gir[y+1][x];

if (yu and as and not(so or sa) or
    so and sa and not(yu or as))
    return;                // çıkmaz sokak: geri dön
```

Anlamı şu: bu karenin üstü *ve* altı zaten ziyaret edilmişse ama sağı *ve* solu boşsa, tahta ikiye bölünmüş demektir ve bir daha birleşemeyiz. O dalda meyve yok, geri dön.

BUDARKEN DİKKAT

Kestiğiniz dalın gerçekten meyve veremeyeceğinden **emin** olmalısınız. Yoksa doğru çözümleri kaybedersiniz ve program hızlı hızlı yanlış yanıt verir — en kötü hata türü. Her budama kuralı için bir kanıt isteyin kendinizden. Yukarıdaki kural için kâğıda birkaç örnek çizmek yetiyor.

ALIŞTIRMALAR

1. **Kümeyi ikiye bölün.** Apple Division: *n* elmayı iki gruba ayırın, ağırlık farkı en az olsun. Hem özyinelemeyle hem bitlerle çözün. (`n ≤ 20` olduğuna dikkat — yukarıdaki kaba kural size ne söylüyor?)
2. **Vezirleri yazdırın.** Yukarıdaki program çözümleri *sayıyor*. Bir de her çözümü tahta olarak ekrana çizsin. 92 çözümün ilk üçüne bakın; birbirlerinin aynadaki hâli mi?
3. **Güzel permütasyon.** CSES 1070: peş peşe gelen öğelerin farkı en az 2 olan bir permütasyon bulun. Önce hepsini üretip eleyin. Sonra `ASIL SORU` doğrudan kurmanın bir yolunu bulun — *n* büyük olduğunda üretmek çalışmaz.
4. **Hamilton patikaları.** CSES 1625: ızgarada sol üstten sağ alta, her kareden bir kere geçen yolları sayın. Önce budamasız yazın ve 6×6'da süreyi ölçün. Sonra yukarıdaki budamayı ekleyin ve tekrar ölçün. Aradaki farkı görün. `ZOR` Başka bir budama kuralı bulabilir misiniz?
5. **Bir kanıt yazın.** Yukarıdaki budama kuralının doğru olduğunu — yani kesilen dalda gerçekten çözüm olamayacağını — birkaç cümleyle anlatın. Kâğıt kalem serbest. Bu alıştırma kod yazmaktan daha değerli.

BÖLÜM V

Çizgeler ve Gezintiler

Şehirler ve yollar, bilgisayarlar ve kablolar, insanlar ve arkadaşlıklar, labirentteki odalar — hepsi aynı matematiksel nesne: **çizge**. Bu bölümde beş satırlık bir gezinme kalıbı öğreneceğiz ve o kalıbı değiştire değiştire yarım düzine ünlü problemi çözeceğiz.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

V.1

Çizge nedir, nasıl saklanır?

Bir **çizge** (*graph*) iki şeyden ibaret: **noktalar** (*vertex*, düğüm de denir) ve onları birleştiren **bağlar** (*edge*, kenar da denir). Hepsi bu. Bu kadar sade bir yapının bu kadar çok şeyi modelleyebilmesi çizge kuramının güzelliği.

Kuramın tarihteki ilk sorusu 1736'da soruldu: Königsberg şehrinde yedi köprü var; hepsinden tam bir kere geçen bir yürüyüş mümkün mü? Euler, mümkün olmadığını kanıtladı — ve bunu yaparken köprülerin uzunluğunun, adaların büyüklüğünün hiç önemli olmadığını, sadece *neyin neye bağlı olduğunun* önemli olduğunu gösterdi. Çizge kuramı o gün doğdu.

C++'ta çizgeyi saklamanın iki yaygın yolu var:

iki temsil

```
// 1) KOMŞULUK MATRİSİ: km[a][b] = a ile b bağlı mı?
const int N = 100;
bool km[N][N]{};
km[1][0] = true;              // 1'den 0'a yönlü bağ
km[0][1] = true;              // yönsüz istiyorsak ikisi de

// ağırlıklı istersek bool yerine sayı:
double agirlik[N][N];
agirlik[3][5] = 2.5;

// 2) KOMŞULUK LİSTESİ: her nokta için komşularının dizisi
using Nokta = unsigned;
vector<vector<Nokta>> komsular;
komsular[1].push_back(2);      // 1'den 2'ye
```

	BELLEK	"A İLE B BAĞLI MI?"	"A'NIN KOMŞULARI?"	NE ZAMAN
Matris	$O(n^2)$	$O(1)$	$O(n)$	Nokta az, bağ çok
Liste	$O(n+b)$	$O(derece)$	$O(derece)$	Nokta çok, bağ seyrek — **çoğu zaman bu**

Yüz bin şehir olan bir soruda matris 10^{10} hücre ister — belleğe sığmaz. Onun için varsayılan tercihiniz komşuluk listesi olsun.

> **PÜF NOKTASI**
>
> Bazen çizgeyi hiç saklamanız gerekmez! Labirent ya da satranç tahtası gibi *ızgara* problemlerinde komşuluk zaten belli: `(y,x)` karesinin komşuları `(y±1,x)` ve `(y,x±1)`. Çizge oradadır, ama görünmez.

V.2

Beş satırlık kalıp

Bu bölümün kalbi burası. Bir çizgeyi gezmenin kalıbı şu — ezberleyin, ömrünüz boyunca kullanacaksınız:

derinlemesine gezi — DFS

```
void gez(Nokta bu) {
  if (gezildi[bu]) return;          // taban: buraya zaten uğradık
  gezildi[bu] = true;
  for (Nokta su : komsular[bu])
    gez(su);
}
```

Beş satır. Ama altında zengin bir temel var; sayalım: `void`, `Nokta`, `bool`, `vector<bool>`, `vector<Nokta>`, `vector<vector<Nokta>>`. Bölüm II'de öğrendiğiniz her şey burada.

Bu yönteme **derinlemesine gezi** (*DFS — depth-first search*) deniyor: bir yola girer, sonuna kadar gider, tıkanınca geri döner. Bölüm IV'teki geri dönüşlü aramanın ta kendisi — sadece adı değişti.

Aynı gezi, üç farklı yol

Özyineleme yerine **yığın** kullanarak da yazabiliriz. Ve işin sihirli yanı: yığını **kuyrukla** değiştirdiğimizde gezi derinlemesineden **enlemesine**ye (*BFS — breadth-first search*) dönüşüyor. Tek kelime fark, bambaşka bir algoritma.

40-gezi.cpp — YIĞIN MI, KUYRUK MU?

```cpp
// derinlemesine, yığınla
void gez2(Nokta ilk) {
  stack<Nokta> tepe;
  tepe.push(ilk);
  while (not tepe.empty()) {
    Nokta bu = tepe.top(); tepe.pop();
    if (gezildi[bu]) continue;
    gezildi[bu] = true;
    for (Nokta su : komsular[bu])
      if (not gezildi[su]) tepe.push(su);
  }
}

// enlemesine, kuyrukla -- yukarıdakiyle neredeyse birebir aynı!
void gez3(Nokta ilk) {
  queue<Nokta> kuyruk;
  kuyruk.push(ilk);
  while (not kuyruk.empty()) {
    Nokta bu = kuyruk.front(); kuyruk.pop();
    if (gezildi[bu]) continue;
    gezildi[bu] = true;
    for (Nokta su : komsular[bu])
      if (not gezildi[su]) kuyruk.push(su);
  }
}
```

Yedi noktalı küçük bir yönlü çizgede üçünü de çalıştırdım. Aynı çizge, üç farklı sıra:

çıktı

```
derinlemesine (özyineleme): 1 2 5 7 3 4 6
derinlemesine (yığınla)   : 1 3 4 7 5 6 2
enlemesine    (kuyrukla)  : 1 2 3 5 4 7 6
```

Enlemesine gezide 1'in bütün komşuları (2 ve 3) hemen başta geliyor. Derinlemesine gezide ise 2'ye girip ta 7'ye kadar gidiliyor, 3 sona kalıyor. İşte bu fark, birazdan en kısa yolu bulmamızı sağlayacak.

GERÇEK BİR HATA

`gez3` içindeki `if (gezildi[bu]) continue;` satırını silerseniz program hâlâ çalışır, ama çıktı şöyle olur:

```
enlemesine (kuyrukla): 1 2 3 5 4 7 6 7
```

7 iki kere yazıldı! Neden? Çünkü aynı nokta kuyruğa *iki farklı yoldan* girebilir; kuyruğa girmekle ziyaret edilmek aynı şey değil. Derinlemesine gezide bu olmuyor — neden olmadığını düşünmek çok öğretici bir alıştırma. Bu hatayı derste yaptık ve ancak çizgeye yedinci noktayı ekleyince fark ettik.

V.3

Kalıbı çalıştırmak: dört problem

1. Kaç oda var?

Bir kat planı verilmiş; `#` duvar, `.` boşluk. Kaç ayrı oda var? Kalıba iki satır ekliyoruz:

ızgarada derinlemesine gezi

```
bool gez(K y, K x, Tahta& t) {
  if (not t[y][x]) return false;    // duvar ya da gezildi
  t[y][x] = false;                  // artık müsait değil
  gez(y-1, x, t);
  gez(y+1, x, t);
  gez(y, x-1, t);
  gez(y, x+1, t);
  return true;                      // bir oda daha bulduk
}

// main içinde: her kareyi dene, kaç kere "true" döndüyse o kadar oda var
```

Tek püf noktası: **etrafı duvarla çevirin.** Tahtayı her yönde bir sıra büyütüp kenarları duvar yapın; böylece `gez(y-1, x)` çağrısı dizinin dışına taşmaz. Bunu yapmazsanız sınır denetimi için dört tane `if` yazmanız gerekir ve birini mutlaka unutursunuz.

2. Labirentte en kısa yol

Aynı ızgara, farklı soru: `A`'dan `B`'ye en kısa yol nedir, tarifi ne? Burada derinlemesine gezi **işe yaramaz** — bulduğu ilk yol en kısası olmak zorunda değil. Kuyruk gerekiyor.

Güzel bir numara var: her kareye “buraya hangi yönden geldim” bilgisini yazarsak, sonunda B'den geriye doğru yürüyerek yolu okuyabiliriz.

43-labirent.cpp — ENLEMESİNE + YOL TARİFİ

```
struct Bilgi { bool yol; char adim; };   // müsait mi, nereden geldik
const Bilgi YOL{true, '.'}, DUVAR{false, '#'};
using Tahta = Dizi<Dizi<Bilgi>>;

queue<Oda> kuyruk;
kuyruk.push(A);
t[A.y][A.x] = {false, 'A'};

while (not kuyruk.empty()) {
  auto [y, x] = kuyruk.front();
  kuyruk.pop();
  if (y == B.y and x == B.x) { vardik = true; break; }

  Dizi<Komsu> komsular{{{y-1,x},'U'}, {{y+1,x},'D'},
                       {{y,x-1},'L'}, {{y,x+1},'R'}};
  for (auto [oda, yon] : komsular)
    if (t[oda.y][oda.x].yol) {
      t[oda.y][oda.x] = {false, yon};   // hem işaretle hem yönü kaydet
      kuyruk.push(oda);
    }
}
```

çıktı

```
##########
#A#.....#
#.#.###.#
#.#...#.#
#.#####.#
#......B#
##########

en kısa yol 10 adım: DDDDRRRRRR
```

NEDEN EN KISA?

Kuyruk, noktaları başlangıca *uzaklık sırasına göre* işler: önce uzaklığı 1 olanlar, sonra 2 olanlar... Öyleyse B'ye ilk vardığımız anda, ondan kısa bir yol olamaz — olsaydı daha önce kuyruktan çıkardı. Bunu tümevarımla düzgünce yazmak güzel bir alıştırma. **Bir çözümün doğru olduğunu bilmekle, doğru olduğunu kanıtlayabilmek ayrı şeyler.**

3. Bağlı parçalar ve yol ağı

n şehir ve bazıları arasında yollar var. Bütün şehirlerin birbirine ulaşabilmesi için en az kaç yeni yol gerek, hangileri?

Çözüm iki adımda: önce **bağlı parçaları** bulun (birbirine ulaşabilen şehir kümeleri), sonra her parçadan bir temsilci seçip onları zincir gibi bağlayın. *k* parça varsa `k-1` yol yeter.

bağlı parçalar

```
vector<unsigned> parca;     // parca[nokta] = kaçıncı parçada

void parcayaEkle(Nokta n, unsigned no) {
  if (parca[n]) return;
  parca[n] = no;
  for (Nokta n2 : cizge[n]) parcayaEkle(n2, no);
}

// main içinde:
unsigned sayi = 0;
vector<Nokta> kok;                    // her parçanın temsilcisi
for (Nokta n = 1; n <= toplam; ++n)
  if (not parca[n]) {
    parcayaEkle(n, ++sayi);
    kok.push_back(n);
  }
// sonra kok[0]-kok[1], kok[1]-kok[2], ... bağla
```

Bütün parçaları bulmak `O(n+b)`, bağlamak `O(n)`. Yani tamamı doğrusal — yüz binlerce şehirde bile anlık.

4. İki takım kurmak

Bir sınıfı iki takıma bölün, ama arkadaş olan iki kişi *farklı* takımlarda olsun. Her zaman mümkün mü? Değil — ve mümkün olup olmadığını anlamanın adı *iki parçalı çizge* (*bipartite graph*) denetimi.

Yine aynı kalıp; sadece “gezildi” yerine “hangi takımda” tutuyoruz ve komşuya karşı takımı veriyoruz:

iki parçalı mı?

```
enum class Takim { Yok, Bir, İki };
vector<Takim> takim;

bool gez(Ogrenci a) {
  for (Ogrenci b : sinif[a])
    if (takim[b] != Takim::Yok) {
      if (takim[b] == takim[a]) return false;   // iki arkadaş aynı takımda!
    } else {
      using enum Takim;
      takim[b] = (takim[a] == Bir) ? İki : Bir;
      if (not gez(b)) return false;
    }
  return true;
}
```

İKİ HATA, İKİ DERS

Bu kodu derste yazarken iki hata yaptık ve `gdb` ile bulduk. Birincisi: `else`'i unutmuştuk — yani taban durumu eksikti, gezi hiç durmuyordu. İkincisi: dış döngüde "bu öğrenci zaten bir takımda mı?" denetimi yoktu. İkisi de aynı ailedendir: **"buraya daha önce geldim mi?" sorusunu sormayı unutmak.** Çizge kodu yazarken ilk denetleyeceğiniz şey budur.

Bir de: dış döngü neden gerekli? Çünkü çizge kopuk olabilir. Tek bir noktadan başlayan gezi, ötekilerin bulunduğu parçaya hiç uğramaz.

V.4

Bir kuyruk, birçok başlangıç

Şimdi bu bölümün en güzel numarası. Ders notlarımızda bir labirent problemi var: içinde canavarlar dolaşıyor, siz çıkışa ulaşmaya çalışıyorsunuz. Canavarlar sizden hızlı değil, ama yolunuzu kesebilirler.

Çözmek için her karenin **en yakın canavara** uzaklığını bilmek gerekiyor. 1000×1000'lik bir tahtada on bine yakın canavar olabilir. Her canavar için ayrı bir gezi mi yapacağız?

yavaş — 28.71 SANİYE

```
for (auto canavar : canavarlar) {   // her canavar için...
  uzaklik[canavar.y][canavar.x] = 0;
  kuyruk.push(canavar);
  while (not kuyruk.empty()) {       // ...bütün tahtayı gez
    // ...
  }
}
```

hızlı — 0.73 SANİYE

```
for (auto canavar : canavarlar) {   // önce HEPSİNİ kuyruğa koy
  uzaklik[canavar.y][canavar.x] = 0;
  kuyruk.push(canavar);
}
while (not kuyruk.empty()) {         // sonra TEK bir gezi
  // ...
}
```

Farkı gördünüz mü? Süslü parantezin yeri değişti, o kadar. Ama birincisi `c × n × m`, ikincisi `c + n × m` hızında çalışıyor. Aynı girdide **28.71 saniye** yerine **0.73 saniye** — kırk kat.

Sezgisel açıklaması çok hoş: bütün canavarları aynı kuyruğa koyduğunuzda *iş bölümü* yapıyorlar. Her canavar sadece kendine en yakın kareleri dolaşıyor; birbirlerine çarptıkları yerde duruyorlar. Buna **çok kaynaklı enlemesine gezi** deniyor ve "her noktanın en yakın *şeye* uzaklığı" tipindeki bütün sorularda işe yarar.

> **ŞUNU DA NOT EDİN**
>
> Hızlı program **100 satır**, yavaş program **141 satır**. Kısa olan hızlıydı. Bu tesadüf değil: kodu sadeleştirirken ne yaptığınızı daha net görürsünüz ve o netlikte daha iyi algoritma saklıdır.

V.5

Yollar eşit değilse: en kısa yol algoritmaları

Şimdiye kadar bütün bağlar eşitti; "en kısa yol" demek "en az adım" demekti. Gerçek hayatta ise bağların bir **ağırlığı** var: kilometre, dakika, ücret, kapasite, hatta ödül. Üç ünlü algoritma, üç farklı durum için.

Dijkstra: bir noktadan hepsine

İstanbul'dan diğer bütün şehirlere en çabuk varan rota nedir? Enlemesine gezi kalıbını alın ve **kuyruğu öncelik sırasıyla değiştirin**. Hepsi bu.

41-dijkstra.cpp — KUYRUK → ÖNCELİK SIRASI

```
using Hat = pair<U, Sehir>;            // {süre, varış} -- SIRA ÖNEMLİ
template <class T> using OncelikSirasi =
    priority_queue<T, vector<T>, greater<T>>;   // küçükten büyüğe

void gez(Sehir ilk, vector<U>& uzaklik) {
  OncelikSirasi<Hat> sira;
  sira.push({0, ilk});
  uzaklik[ilk] = 0;

  while (not sira.empty()) {
    auto [u, bu] = sira.top();
    sira.pop();
    if (uzaklik[bu] < u) continue;          // eski, işi bitmiş kayıt

    for (auto [sure, su] : hatlar[bu]) {
      U yeni = u + sure;
      if (yeni < uzaklik[su]) {
        uzaklik[su] = yeni;
        sira.push({yeni, su});
      }
    }
  }
}
```

İki ayrıntı hayati. Birincisi: `pair` içinde **süre önce, şehir sonra** — çünkü öncelik sırası soldan karşılaştırır ve biz en yakın şehri istiyoruz. İkincisi: `greater<T>`. Varsayılan `priority_queue` en *büyüğü* verir; biz en küçüğü istiyoruz. (`greater` bir *işlevci* — nesnesi işlev gibi çağrılabilen bir tür.) Bir de `continue` satırı: aynı şehir sıraya birkaç kere girebilir, eski kayıt gelince atlıyoruz. Bu, Dijkstra'nın *tembel* sürümü; klasik sürüm sıradan çıkan şehri bir `bitti` dizisinde işaretler ve ona bir daha dokunmaz. İkisi eksi ağırlıkta farklı davranır; alıştırmada göreceksiniz.

çalışırken

```
girdi:  3 şehir, 4 tek yönlü hat
        1 -> 2 : 6      1 -> 3 : 2
        3 -> 2 : 3      1 -> 3 : 4

çıktı:  0 5 2
        // 1'den 2'ye doğrudan 6, ama 3 üzerinden 2+3 = 5!
```

Hızı `O(b log b)`: her bağın üzerinden bir kere geçiyoruz (`O(b)`) ve her seferinde öncelik sırasına ekleme yapıyoruz (`O(log b)`).

Floyd–Warshall: herkesten herkese

Bütün şehir çiftleri arasındaki uzaklığı istiyorsanız, Dijkstra'yı *n* kere çalıştırabilirsiniz — ya da şu üç satırı yazabilirsiniz:

42-floyd.cpp — ÜÇ DÖNGÜ, O KADAR

```
for (int o = 1; o <= n; ++o)          // o: "ortadaki durak"
  for (int a = 1; a <= n; ++a)
    for (int b = 1; b <= n; ++b)
      f[a][b] = min(f[a][b], f[a][o] + f[o][b]);
```

Fikri şu: *a*'dan *b*'ye giderken *o* şehrine uğrarsam daha mı kısa olur? En dıştaki döngü, izin verilen ara durakları birer birer artırıyor. Bu tam anlamıyla bir dinamik program — kanıtı Bölüm VI'nın konusu.

çıktı — 4 ŞEHİR, 4 ÇİFT YÖNLÜ YOL

```
yollar: 1-2:5   1-4:9   2-3:2   3-4:7

        1   2   3   4
      ---------------
  1 |   0   5   7   9      // 1'den 3'e: 5+2 = 7
  2 |   5   0   2   9      // 2'den 4'e: 2+7 = 9
  3 |   7   2   0   7
  4 |   9   9   7   0
```

Bellman–Ford: eksi ağırlıklar da olabilir

Bir odadan ötekine tünellerden geçiyorsunuz; bazı tüneller ödül veriyor, bazıları ceza. En büyük ödülü nasıl toplarsınız? Dijkstra burada **yanlış yanıt verir** — çünkü o "aceleci" (*greedy*) bir algoritma: bir şehri işini bitmiş sayıp bir daha bakmaz. Eksi ağırlıklı bir bağ, o kararı sonradan bozabilir.

Bellman-Ford — N-1 TUR

```
vector<S> skor(n+1, -SONSUZ);
skor[1] = 0;

for (K k = 1; k < n; ++k)                    // en fazla n-1 tünel gerekir
  for (auto [a, b, x] : tuneller) {
    if (skor[a] == -SONSUZ) continue;        // ulaşılamayan şehirden bağ çıkmaz
    skor[b] = max(skor[b], skor[a] + x);
  }

// bir tur daha atıp hâlâ iyileşen var mı diye bakarsak,
// sonsuza kadar artan bir ÇEVRİM bulmuş oluruz:
for (auto [a, b, x] : tuneller)
  if (skor[a] != -SONSUZ and skor[a] + x > skor[b])
    cevrim.push_back(b);
```

İki küçük tuzak. `SONSUZ` için türün en büyük değerini seçmeyin; `1e18` gibi büyük ama toplayınca taşmayan bir sayı seçin. Ve ulaşılamayan bir şehirden çıkan bağı atlayın: yoksa `-SONSUZ + x` “neredeyse sonsuz” bir skor olur, ulaşılamayan şehir ulaşılmış görünür.

Neden `n−1` tur? Çünkü en uzun basit yol en fazla `n−1` bağ içerir. Ve `n`'inci turda hâlâ bir şey iyileşiyorsa, bu ancak sonsuza kadar dönülebilen kârlı bir çevrim varsa olur. Aynı kodla hem yolu buluyor hem de “sonsuz kâr” durumunu yakalıyoruz — çok zarif.

Hangisini seçmeli?

ALGORİTMA	NE BULUR	HIZ	EKSİ AĞIRLIK
Enlemesine gezi	Bir noktadan hepsine, *bağlar eşitse*	`O(n+b)`	—
Dijkstra	Bir noktadan hepsine	`O(b log b)`	Hayır
Bellman–Ford	Bir noktadan hepsine + çevrim tespiti	`O(n·b)`	**Evet**
Floyd–Warshall	Herkesten herkese	$O(n^3)$	**Evet**

Bu tabloyu ezberlemeyin; anlayın. Sorudaki üç şeye bakın: bağlar eşit mi, ağırlıklar eksi olabilir mi, kaç başlangıç noktası var? Yanıt sizi doğru satıra götürür.

ALIŞTIRMALAR

1. **Odaları sayın.** CSES 1192. Derinlemesine geziyle çözün, sonra bir de yığınla yazın. Hangisi daha kolay okunuyor?

2. **Labirentte yol.** CSES 1193. Yukarıdaki yol tarifi numarasını kendiniz kurun.

3. **Yol ağı tasarımı.** CSES 1666. Bağlı parçaları önce özyinelemeyle, sonra `std::stack` ile bulun. Büyük girdide özyinelemeli sürüm neden çökebilir?

4. **İki takım.** CSES 1668. Sonra şunu düşünün: mümkün olmadığı durumda çizgede ne var? İPUCU Tek sayıda bağdan oluşan bir çevrim.

5. **Güzel bir çevrim bulun.** CSES 1669. Gezi kalıbına "bir önceki nokta" girdisini ekleyin ki `A→B→A` gibi sahte çevrimleri saymayın.

6. **Üç yol problemi.** CSES'in 1671, 1672 ve 1673 soruları sırasıyla Dijkstra, Floyd–Warshall ve Bellman–Ford ister. Üçünü de yazın. ASIL İŞ Sonra üçünü tek bir sayfada karşılaştırın: hangi satır hangisini ötekinden ayırıyor?

7. **Dijkstra'yı kırın.** Önce klasik sürümü yazın: sıradan çıkan şehri bir `bitti` dizisinde işaretleyin ve bitmiş şehre bir daha dokunmayın. Sonra eksi ağırlıklı bir bağ içeren, *en fazla dört noktalı* bir çizge çizin; öyle ki bu sürüm yanlış yanıt versin. Bulduğunuzda algoritmanın neden aceleci olduğunu gerçekten anlamış olacaksınız. Yukarıdaki tembel sürüm aynı çizgede doğru yanıtı verir, çünkü eski kaydı atlar ama şehri yeniden açar; onu kırmak için eksi toplamlı bir *çevrim* gerekir, o zaman hiç durmaz. Deneyin.

BÖLÜM VI

Dinamik Programlama

Korkutucu bir adı var ama arkasındaki fikir bir cümle: **büyük problemi kendinden küçük parçalarına böl, her parçayı bir kere çöz, bir daha çözme.** Bu bölümde o cümleyi bir reçeteye çevireceğiz — ve kitapçığı, bundan sonra nereye gideceğinizi konuşarak kapatacağız.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

VI.1

Aslında bunu zaten yaptınız

Bölüm IV'te Fibonaççi'ye tek satırlık bir bellek ekleyip kırk bin kat hızlandırmıştık. İşte o, dinamik programlamaydı. Adı büyük ama yaptığı iş bu kadar.

Dinamik programın iki yazılış biçimi var, ikisini de bilmek gerekiyor:

Yukarıdan aşağı (YA)
Özyinelemeli işlev + bellek. Tanımı doğrudan koda çevirirsiniz; **düşünmesi ve kanıtlaması kolay**. Derin çağrılarda yığın taşabilir.

Aşağıdan yukarı (AY)
En küçükten başlayıp bir çizelgeyi doldurursunuz. Özyineleme yok, yığın sorunu yok, **genelde daha hızlı** ve belleği kısmak mümkün.

Pratikte iyi bir sıra şudur: **YA ile düşün, AY ile yaz.** Önce özyinelemeli tanımı bulun (çünkü orada kanıt var), sonra onu döngüye çevirin.

VI.2

Izgarada kaç yol var?

Project Euler'in 15. sorusu: 20×20'lik bir ızgaranın sol üst köşesinden sağ alt köşesine, sadece sağa ve aşağıya giderek kaç farklı yol vardır?

Özyinelemeli tanım çok kolay: `(r, c)` karesine ya üstten ya soldan gelinir. Öyleyse:

1. yol — BELLEKSİZ

```
Sayi yol1(int r, int c) {
  if (r == 0 or c == 0) return 1;      // kenardaysak tek yol var
  return yol1(r-1, c) + yol1(r, c-1);
}
```

Doğru, ama 12×12'de bile beş milyondan fazla çağrı yapıyor; 20×20'yi hiç bitiremez. Belleği ekleyelim:

2. yol — BELLEKLİ

```
map<pair<int,int>, Sayi> bellek;

Sayi yol2(int r, int c) {
  if (r == 0 or c == 0) return 1;
  auto anahtar = make_pair(r, c);
  if (bellek.contains(anahtar)) return bellek[anahtar];
  return bellek[anahtar] = yol2(r-1, c) + yol2(r, c-1);
}
```

12×12 IZGARA	SONUÇ	ÇAĞRI SAYISI
Belleksiz	2 704 156	5 408 311
Bellekli	2 704 156	289

Şimdi aynı şeyi aşağıdan yukarı yazalım. Özyineleme yok, sadece bir çizelge dolduruyoruz:

3. yol — AŞAĞIDAN YUKARI

```
Sayi yol3(int n) {
  vector<vector<Sayi>> t(n+1, vector<Sayi>(n+1, 1));  // kenarlar 1
  for (int r = 1; r <= n; ++r)
    for (int c = 1; c <= n; ++c)
      t[r][c] = t[r-1][c] + t[r][c-1];
  return t[n][n];
}
```

Ve son bir güzellik: çizelgenin sadece *bir önceki satırına* bakıyoruz. Öyleyse bütün tabloyu saklamaya gerek yok — n^2 bellek yerine n yeter:

4. yol — TEK SATIR BELLEK

```
Sayi yol4(int n) {
  vector<Sayi> satir(n+1, 1);
  for (int r = 1; r <= n; ++r)
    for (int c = 1; c <= n; ++c)
      satir[c] += satir[c-1];   // satir[c]'nin eski değeri = üstteki satır!
  return satir[n];
}
```

```
20×20 ızgarada yol sayısı: 137 846 528 820
```

Dört yol, aynı yanıt. Bir problemi dört farklı biçimde yazabilmek, onu gerçekten anlamış olmanın en iyi göstergesi.

> **DENEYİN**
>
> Dördüncü sürümdeki `satir[c] += satir[c-1];` satırı neden çalışıyor? Kâğıda 3×3'lük bir çizelge çizin ve döngüyü elle işletin. Bu tek satırı anladığınızda dinamik programlamanın bellek hilelerinin çoğunu anlamış olursunuz.

VI.3

Beş adımlık reçete

Bir problemi gördüğünüzde dinamik programa uygun olup olmadığını nasıl anlarsınız, uygunsa nasıl kurarsınız? Her seferinde aynı beş adım:

1. **Aradığınız şeyi bir fonksiyon olarak yazın.** "$f(x)$ = x toplamına varan en kısa dizinin boyu." Bu adımı atlamayın; en zor ve en önemli adım budur.
2. **Taban durumlarını belirleyin.** $f(0)$ kaç? Anlamsız girdilerde (eksi sayılar gibi) ne olsun? Genellikle 0, 1 ya da sonsuz.
3. **Tümevarım formülünü kurun.** $f(x)$'i kendinden küçük değerlerle yazın. Sorunuz şu: "son adımda ne yaptım?"
4. **Kanıtlayın.** İki soru: formül bütün durumları kapsıyor mu? Hiçbir durumu iki kere saymıyor mu?
5. **Kodlayın.** Ya YA + bellek, ya da AY + çizelge.

Şimdi bu reçeteyi üç soruya uygulayalım. Üçü de bozuk para ve zarlarla ilgili, üçü de aldatıcı biçimde benzer — ve tam da bu yüzden çok öğretici.

Soru 1: Zar dizilerini sayalım

Bir zarı defalarca atıyoruz. Toplamı n olan kaç farklı zar dizisi var? (CSES DP 1)

1. Fonksiyon: *f(n)* = toplamı *n* olan dizi sayısı.
2. Taban: *f*(0) = 1 — boş dizinin toplamı sıfır, ve o da bir dizidir.
3. Formül: son atılan zar 1, 2, 3, 4, 5 ya da 6 olabilir:

tümevarım

```
f(n) = f(n-1) + f(n-2) + f(n-3) + f(n-4) + f(n-5) + f(n-6)
```

4. Kanıt: Her dizinin bir son zarı var, o da altı değerden biri — yani bütün durumlar kapsandı. Ve son zarı farklı olan iki dizi aynı olamaz — yani hiçbir şey iki kere sayılmadı. İki koşul da sağlandı.

52-zar.cpp AŞAĞIDAN YUKARI

```
vector<S> f(n+1, 0);
f[0] = 1;
for (int x = 1; x <= n; ++x)
  for (int zar = 1; zar <= 6; ++zar)
    if (x - zar >= 0) f[x] = (f[x] + f[x - zar]) % MOD;
```

```
f(0)=1  f(1)=1  f(2)=2  f(3)=4  f(4)=8  f(5)=16  f(6)=32  f(7)=63
f(8)=125
```

İlk yedi değer ikinin kuvvetleri gibi gidiyor, sonra *f*(7)=63 olunca bozuluyor. Neden? Çünkü 7'ye kadar bütün eski değerleri toplayabiliyorduk, 7'de birincisi (f(0)) listeden düşüyor. Böyle küçük gözlemler formülünüzün doğru olduğuna dair en hızlı sağlamadır.

Soru 2: En az kaç bozuk para?

Paralarımız 1, 5 ve 7 kuruş. 11 kuruşu en az kaç parayla öderiz? (CSES DP 2)

> **ACELECİLİK NEDEN YETMEZ?**
>
> Sezgi "en büyükten başla" der: 7 + 1 + 1 + 1 + 1 = **5 para**. Oysa en iyi çözüm 5 + 5 + 1 = **3 para**. *Aceleci* (*greedy*; birebir çevirisiyle "açgözlü") yöntem burada yanılıyor — acele işe şeytan karışır. Bu yüzden acele etmiyor, bütün olasılıkları küçük parçalara bölüyoruz.

Formül: Son koyduğum para hangisiydi? Hepsini deneyip en iyisini alalım:

tümevarım

```
f(x) = 1 + min { f(x-b₁), f(x-b₂), ..., f(x-bₙ) }

f(0) = 0
f(x) = ∞     (x < 0 için)
```

Eksi girdilerde “sonsuz” demek şık bir numara: `min` onları kendiliğinden eliyor, biz de her para değeri için ayrı koşul yazmaktan kurtuluyoruz.

51-bozukpara.cpp

```
vector<S> f(hedef+1, SONSUZ);
f[0] = 0;
for (int x = 1; x <= hedef; ++x)
  for (S p : paralar)
    if (x - p >= 0) f[x] = min(f[x], 1 + f[x - p]);
```

Soru 3: Diziler mi, kümeler mi?

Şimdi en ince ayrım. Paralarımız 2, 3, 5 ve hedef 9. İki soru:

- Toplamı 9 olan kaç **dizi** var? Yani sıra önemli: `2+2+5` ile `5+2+2` ayrı sayılıyor. (CSES DP 3)
- Toplamı 9 olan kaç **çoklu küme** var? Sıra önemsiz ama tekrar sayılır: `2+2+5` ile `5+2+2` aynı, ikisi de `{2,2,5}`. (Bölüm II'nin kümesi tekrarı yutardı; buradaki yutmuyor, o yüzden “çoklu”.) (CSES DP 4)

DİZİLER (8 TANE)	ÇOKLU KÜMELER (3 TANE)
`2+2+5`, `2+5+2`, `5+2+2`, `2+2+2+3`, `2+2+3+2`, `2+3+2+2`, `3+2+2+2`, `3+3+3`	`{2,2,5}`, `{2,2,2,3}`, `{3,3,3}`

Şimdi çok güzel bir şey olacak. İki kodun farkına bakın:

51-bozukpara.cpp — TEK FARK: DÖNGÜ SIRASI

```
// DİZİLER: sıra önemli
d[0] = 1;
for (int x = 1; x <= hedef; ++x)      // önce hedef
  for (S p : paralar)                 //   sonra para
    if (x - p >= 0) d[x] += d[x - p];

// KÜMELER: sıra önemsiz
k[0] = 1;
for (S p : paralar)                   // önce para
  for (int x = p; x <= hedef; ++x)    //   sonra hedef
    k[x] += k[x - p];
```

```
kaç farklı dizi : 8   |   kaç farklı küme : 3
```

İki döngünün yerini değiştirdik, cevap 8'den 3'e düştü. Neden? Çünkü ikinci kodda paraları *sırayla* ele alıyoruz: önce bütün 2'leri, sonra 3'leri, sonra 5'leri. Böylece her küme yalnızca **bir** sırada üretiliyor, tekrarlar kendiliğinden yok oluyor.

TAKIM İŞİ

Bu iki kodu ayrı ayrı elle işletin — biriniz birincisini, öteki ikincisini, `hedef = 5` ve `paralar = {2, 3}` için. Her adımda dizinin içeriğini kâğıda yazın. Sonra karşılaştırın. Bu yarım saat, dinamik programlamada saatlerce okumaktan daha çok şey öğretir.

VI.4

Kanıt neden bu kadar önemli?

Bir dinamik program yazdığınızda, doğru olup olmadığını test ederek anlayamazsınız — çünkü yanlış bir formül de küçük örneklerde doğru yanıt verebilir. Elinizde bir kanıt olmalı.

İyi haber şu: kanıtlar genelde iki cümle. Floyd–Warshall'ı hatırlayın (Bölüm V). *f(a,b,k)*, *a*'dan *b*'ye giden ve yalnızca `{1..k}` kümesindeki şehirlerden geçen en kısa yol olsun. O zaman:

Floyd-Warshall kanıtı

```
f(a, b, 0) = h(a, b)                    // hiç ara durak yok: doğrudan yol

f(a, b, k) = min( f(a, b, k-1),         // k şehrine UĞRAMAZSAK
                  f(a, k, k-1) + f(k, b, k-1) )   // UĞRARSAK
```

Kanıtın tamamı tek cümle: *a*'dan *b*'ye giderken *k* şehrine ya uğrarız ya uğramayız — üçüncü bir ihtimal yok. Bütün durumlar kapsandı, hiçbiri iki kere sayılmadı. Bitti.

Bu formülü gördüğünüzde `k`'nın en dış döngü olması gerektiği de kendiliğinden anlaşılıyor — işte kanıtın kodu nasıl yazacağınızı söylemesi budur. Bölüm V'teki üç satırlık Floyd–Warshall kodu, bu formülün doğrudan çevirisi.

Yarım bıraktığımız bir problem

İki yazının en uzun **bitişik ortak altdizisi**ni (*longest common substring*) bulmak, çizelgeli dinamik programlamanın en klasik örneklerinden. `tane[k1][k2]`, birinci yazının *k1*, ikinci yazının *k2* konumunda biten bitişik ortak altdizinin uzunluğu olsun. (Yarışma dilindeki "en uzun ortak altdizi", *LCS*, bitişiklik istemez; onun çizelgesi başka. O da size kalsın.)

altdizi.cpp — SİZE BIRAKIYORUM

```cpp
using yazi = std::string;

yazi bul2(const yazi& y1, const yazi& y2) {
    const int boy1 = y1.size(), boy2 = y2.size();
    int enUzun = 0, k = 0;   // şu ana kadarki en uzun ve nerede bittiği

    std::vector<std::vector<int>>
        tane(boy1 + 1, std::vector<int>(boy2 + 1, 0));

    for (int k1 = 0; k1 < boy1; ++k1) {
        for (int k2 = 0; k2 < boy2; ++k2) {
            if (y1[k1] == y2[k2]) {
                int m = tane[k1+1][k2+1] = tane[k1][k2] + 1;
                // ... buradan sonrası size kalıyor
            }
        }
    }
    if (enUzun > 0) return y1.substr(k + 1 - enUzun, enUzun);
    else return yazi{""};
}
```

Bu işlevi **bilerek** yarım bıraktım — ders notlarımızda da öyle duruyor. Eksik olan birkaç satır. Ama önce şu denemeleri yazın, sonra doldurun:

önce testler

```cpp
void dene(yazi y1, yazi y2, yazi ortak) {
    assert(bul2(y1, y2) == ortak);
}

dene("abca",      "daxbcby",    "bc");
dene("abc",       "xyz",        "");
dene("xyabc",     "xydabca",    "abc");
dene("abcdxyad",  "xyabcdxyb",  "abcdxy");
dene("123",       "345",        "3");
dene("1010110010","10110",      "10110");
dene("1110111011","10011",      "011");
```

Testleri *önce* yazmanın faydası şu: çözümü kodlamadan önce problemi tam olarak anlamak zorunda kalıyorsunuz. Yedi test yazıp hepsini geçen bir işleve, gözle bakıp “doğru görünüyor” dediğiniz bir işlevden çok daha fazla güvenebilirsiniz.

SON SÖZ

Yola devam

Altı bölüm bitti. Geriye dönüp bakın: bir değişken tanımlamayı bilmiyordunuz, şimdi Bellman–Ford algoritmasının neden `n−1` tur döndüğünü anlatabiliyorsunuz. Bu, bir dönemde kat edilebilecek epey uzun bir yol.

Bundan sonrası artık ders değil, **alışkanlık**. Şunları öneriyorum:

Haftada üç soru

CSES Problem Set sırayla çözülmek için hazırlanmış. Baştan başlayın, atlamayın. Haftada üç soru, bir yılda 150 soru eder.

Kitabı okuyun

Antti Laaksonen'in *Rekabetçi Programlama El Kitabı*, Türkçesi de var. Ders deposunda `kitap/` dizininde. Anlamadığınız yerleri atlayarak okuyun, sonra dönün.

Yarışmalara girin

Codeforces ve AtCoder'da haftalık yarışmalar var. Sıralamanız önemli değil; iki saat boyunca zorlanmak önemli.

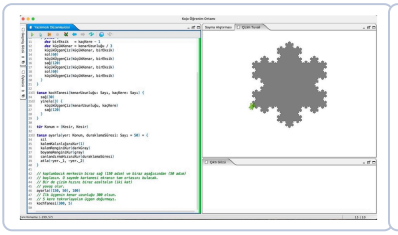
Bir şey inşa edin

Algoritma sorusu olmayan bir şey: bir oyun, bir çizim programı, bir simülasyon. Yarışma kodu bir günde ölür; proje kodu sizinle kalır.

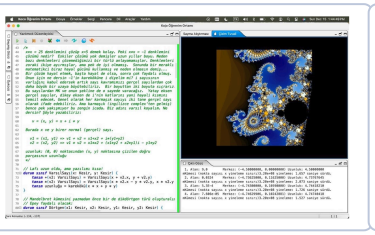
Başka ortamlar, başka keyifler

C++ hızlı ve sağlam, ama tek dil değil — ve bazı şeyleri anlamanın yolu onları *çizdirmekten* geçiyor. Derste zaman zaman bahsettiğim Koco tam da bunun için: Scala diliyle yazıyorsunuz, yazdığınız her satırın sonucunu anında ekranda görüyorsunuz.

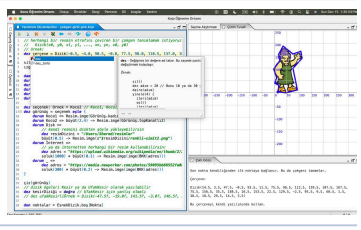
Aşağıdakiler o ortamda yazılmış birkaç program. Üçü de bu kitapçıkta gördüğünüz kavramların üstüne kurulu:



Koch tanesi. Kuralı tek cümle: her kenarın ortasına bir üçgen koy — sonra o üçgenin kenarlarına da. Bölüm IV'teki özyinelemenin ta kendisi, bu sefer gözle görülür hâli.



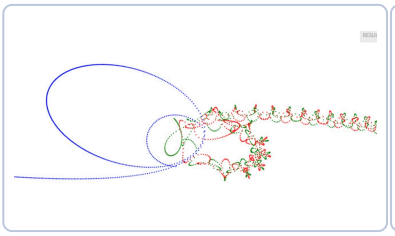
Mandelbrot kümesi. Karmaşık sayılar üzerinde tek satırlık bir kural: `z ← z² + c`. Bu kadarcık kuraldan sonsuz derinlikte bir desen çıkıyor; sınırı hem var hem yok.



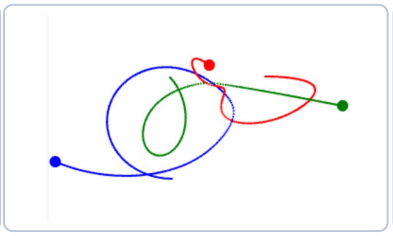
Çokgen çerçeve. Herhangi bir resmin etrafına, kenar sayısını siz seçerek çerçeve çizen bir program. Geometri, döngü ve biraz şaka.

Aynı program, üç farklı başlangıç

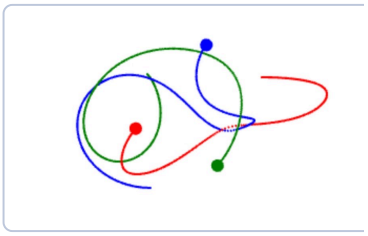
Şu üçü ise **üç cisim problemi**: birbirini çeken üç gök cismi. Newton'un yasaları üç satır tutuyor, ama üç cisim için kalem kâğıtla kapalı bir çözüm *yok* — tek yol adım adım benzetim (*simülasyon*). Üç çizim de aynı programın çıktısı; aralarındaki tek fark başlangıç konum ve hızları.



Birinci deneme



İkinci deneme



Üçüncü deneme

Minicik bir fark, bambaşka bir yörünge. Buna *kaosa duyarlılık* deniyor ve neden hava durumunu on gün öteye tahmin edemediğimizin de cevabı bu. Programınız tamamen belirlenimci (*deterministic*) — her çalıştırışta aynı girdiye aynı çıktıyı veriyor — ama yine de sonucunu önceden kestiremiyorsunuz. Çalıştırmadan bilinemeyen şeyler var; bilgisayarın en güzel tarafı da bu.

> **DENEYİN**
>
> Koco'yu kurun ve Mandelbrot kümesine dalın; sonra Koch tanesini kendiniz yazın. Sadece birkaç satır sürüyor. Kurulum ve ilk adımlar burada. Ardından aynı iki çizimi C++'la yapmayı deneyin — iki dili yan yana görmek ikisini de daha iyi öğretir.

Nereye gidilir?

Bu kitapçıkta değinemediğimiz ama sırada bekleyen konular: **ayrık küme birleşimi** (*union-find*), **dilim ağaçları** (*segment tree*), **kapsayan ağaçlar** (*minimum spanning tree*), **bit işlemleri**, **sayı kuramı** ve **hesaplama geometrisi**. Hepsinin girişi ders deposunda var.

Bir de büyük sorular var. Kümeyi ikiye bölme problemini Bölüm IV'te `O(2ⁿ)` ile çözdük. Daha hızlısı var mı? Kimse bilmiyor — bu, matematiğin en ünlü açık problemi olan **P ile NP** sorusuna bağlı. Yani bir lise alıştırmasından, insanlığın çözemediği bir soruya iki adımda varıyorsunuz. Bilgisayar biliminin güzelliği tam olarak burada.

Son üç cümle

Bu kitapçık boyunca hataları sakladım: taşan sayılar, iki kere kuyruğa giren düğümler, unutulan taban durumları, kırk kat yavaş çalışan programlar. Çünkü hepsi gerçekti ve hepsini birlikte bulduk. **Hata yapmak öğrenmenin yan etkisi değil, yöntemidir.**

Takıldığınızda yalnız kalmayın. Bir arkadaşınızı bulun, problemi ona anlatın, kâğıt kalemle küçük bir örneği elle çözün. Bu kitapçığın en çok tekrar ettiği tavsiye buydu ve tesadüf değil.

Ve son olarak: acele etmeyin. Uzun ince bir yoldayız; yolda yürümenin kıymetini bilmek ve tadını çıkarmak da işin bir parçası. Kolay gelsin, keyifli ve öğretici olsun.

SON ALIŞTIRMALAR

1. **Dört DP sorusu.** CSES DP bölümünün ilk dördü (1633, 1634, 1635, 1636). Her biri için önce beş adımlık reçeteyi kâğıda yazın, sonra kodlayın.
2. **Ortak altdiziyi tamamlayın.** Yukarıdaki `bul2()` işlevini bitirin. Yedi testin hepsi geçsin. `DEVAMI` Aynı problemi yukarıdan aşağı (özyineleme + bellek) yöntemiyle de çözün.
3. **Zıplayan kurbağa.** AtCoder DP A. Klasik bir başlangıç sorusu; reçeteyi uygulayın.
4. **Engelli ızgara.** AtCoder DP H. Bu bölümdeki ızgara probleminin engelli hâli. Tek satır değişecek.
5. **Kendi kitapçığınızı yazın.** Öğrendiğiniz bir konuyu, bir arkadaşınıza anlatacak biçimde iki sayfa yazın. Kod örnekleri koyun, çalıştırın, çıktıyı yapıştırın. Bir şeyi anlatabildiğiniz gün onu gerçekten öğrenmiş olursunuz. `EN ÖNEMLİSİ`

Bu kitapçık 2024–25 ve 2025–26 dönemi ders notlarından, sınıfta birlikte yazdığımız programlardan ve sorduğunuz sorulardan derlendi. İçindeki bütün programlar derlenip çalıştırıldı; çıktılar gerçek çıktılardır. Ders notlarının tamamı ve daha fazlası GitHub deposunda.